

MittiMitra Ultimate Guide

Team A-14 | Amrita Vishwa Vidyapetham

2026

Contents

MITTIMITRA ULTIMATE GUIDE	6
Team A-14, Amrita Vishwa Vidyapetham	6
TABLE OF CONTENTS	6
1. PROJECT IDENTITY	6
2. WHAT MITTIMITRA DOES — THE BIG PICTURE	7
3. TECHNOLOGY STACK	7
4. BUILD SYSTEM — EVERY GRADLE FILE EXPLAINED	9
4a. Project-Level build.gradle	9
4b. App-Level build.gradle (THE CRITICAL ONE)	10
4c. settings.gradle	12
4d. gradle.properties	13
4e. proguard-rules.pro — Full Rules Explained	13
5. ANDROIDMANIFEST.XML — FULL BREAKDOWN	14
Permissions Declared	14
Application Attributes	15
Activity List (26 Activities)	15
FileProvider	15
6. APPLICATION LIFECYCLE — MittiMitraApp.java	16
7. ALL 26 ACTIVITIES — FUNCTION BY FUNCTION	16
BaseActivity.java	16
SplashActivity.java	17
LoginActivity.java	17
SignupActivity.java	18
WelcomeActivity.java	18
ForgotPasswordActivity.java	18
HomeActivity.java	18
ScanActivity.java	19

RecommendationActivity.java	21
HistoryActivity.java	21
CompareActivity.java	22
TipActivity.java (Kisan Sahayak)	22
ProfileActivity.java	22
AccountSecurityActivity.java	23
DocumentsActivity.java	23
SettingsActivity.java	23
LanguageActivity.java	24
ThemeSettingsActivity.java	24
ManageDataActivity.java	24
NotificationSettingsActivity.java	24
PlantScanActivity.java	25
WeatherAlertsActivity.java	25
CropCalendarActivity.java	25
IrrigationActivity.java	25
MandiActivity.java	26
SchemeActivity.java	26
FarmTaskPlannerActivity.java	26
UnitConverterActivity.java	27
ContactActivity.java	27
HelpActivity.java	27
LinkEmailActivity.java	27
AccessibilityActivity.java	27
8. DATABASE ARCHITECTURE — ROOM v8	27
Overview	27
Database Class: MittiMitraDatabase.java	27
Migration History	28
Entities (9 tables)	28
DAOs (9 interfaces)	29
9. NETWORK LAYER — ALL 6 API SERVICES	30
RetrofitClient.java	30
SoilApiService.java	30
OpenMeteoService.java	31
GeocodingService.java	31
MandiApiService.java	32
GroqApiService.java	32
ElevationApiService.java	32
10. FIREBASE SERVICES	33
Firebase Auth	33
Firestore	33
Firebase Storage	33
Firebase Crashlytics	34
Firebase Analytics	34
Firebase App Check	34

Firebase Cloud Functions	34
11. ML / AI PIPELINE	35
TensorFlow Lite — On-Device Soil Classification	35
Groq Llama 3.3 70B — Natural Language Recommendations	35
HuggingFace — Plant Disease Detection	35
12. BACKGROUND PROCESSING — NotificationWorker.java	36
13. REPOSITORY PATTERN — DATA LAYER	37
Overview	37
Domain Interfaces	37
Implementations	37
14. SECURITY ARCHITECTURE	38
Layers of Security	38
Authentication Flow Security	39
15. UTILITY CLASSES	39
SessionManager.java	39
AppPreferences.java	39
UserIdentityResolver.java	39
NutrientStatus.java	39
SoilNutrientMapper.java	40
BitmapUtils.java	40
ImageCompressor.java	40
NetworkUtils.java	40
AnalyticsHelper.java	40
CrashlyticsHelper.java	40
SchemeRepository.java	40
SoilDataManager.java	41
ErrorHandler.java	41
Result.java	41
WeatherUtils.java (referenced but path not shown)	41
LocationContextResolver.java	41
16. LOCALIZATION — 14 LANGUAGES	41
Implementation	41
17. DATA FLOW — END TO END (SCAN TO RECOMMENDATION)	42
18. IMPORTANT APIs USED AND WHY	43
ISRIC SoilGrids v2.0	43
Open-Meteo	43
data.gov.in / Agmarknet	44
Groq Cloud (Llama 3.3 70B)	44
HuggingFace (via Backend)	44
TensorFlow Lite (Local ML)	44
Firebase Cloud Functions	44

Firebase App Check with Play Integrity	44
19. DEPLOYMENT — BUILD, SIGN, RELEASE	44
Local Debug Build	44
Local Release Build	45
Release AAB (for Play Store)	45
Keystore	45
Firebase Console Setup	45
GitHub Actions CI/CD	45
Versioning	46
20. FUNCTION-TO-FUNCTION MAPPING	46
User Scan Flow (complete call chain)	46
Notification Flow	47
21. IMPORTANT FUNCTIONS LIST	48
22. PROJECT NUANCES AND GOTCHAS	49
1. Two Package Names	49
2. Kotlin Plugin on Java Project	49
3. SoilGrids JSON Array — The Critical Fix	49
4. JSON Key Mismatch Bug (Fixed)	49
5. LinearLayoutManager Before setAdapter (Two Crashes)	49
6. GPS Precision Loss Bug (Fixed)	49
7. Room PK Field Name	49
8. Thread Safety in ScanActivity	50
9. Duplicate WorkManager Scheduling (Fixed)	50
10. Phone Uniqueness Enforcement	50
11. Firebase App Check + Emulator	50
12. Firestore Field Locked at Create	50
13. TFLite + Main Thread (ANR Fix)	50
14. TextToSpeech Locale	50
15. Notification ID Overflow	50
16. Dark Mode Auto-Switching	51
17. Firestore Offline Mode + PersistentCacheSettings	51
18. Guest User Architecture	51
19. Yoda Conditions	51
20. try-with-resources Enforcement	51
23. VIVA QUESTIONS PREP	51
APPENDIX: KEY STRING RESOURCES	53
APPENDIX: FIRESTORE SECURITY RULES SUMMARY	54
APPENDIX: ROOM DATABASE MIGRATION SEQUENCE	54
24. DEEP CODE — RecommendationActivity	54
Full Function Breakdown	54

25. DEEP CODE — BackendFunctionsClient	56
Architecture	56
Functions Exposed	56
callEndpoint() — The Generic Dispatcher	56
toCallablePayload()	57
parseEnvelope()	57
mapThrowableToEnvelope()	57
StringOrListAdapter	57
ApiEnvelope.java	57
26. DEEP CODE — FirebasePredictionRepository	58
Fallback Decision Logic	58
Soil Advisory Path	58
Plant Diagnosis Path (More Complex)	58
fetchPlantDiagnosisWithLocalGroq()	58
parseGroqDiagnosisResponse()	58
extractFirstJsonObject(String rawText)	59
27. DEEP CODE — TaskSuggestionEngine	59
Exact Thresholds for Task Creation	59
28. DEEP CODE — AiModels DTOs (Full)	59
SoilAdvisoryRequest (sent to Cloud Function)	59
SoilAdvisoryData (received from Cloud Function)	60
PlantDiagnosisRequest	60
PlantDiagnosisData	60
CropScheduleRequest	60
CropScheduleData	60
ScheduleItem	60
ChatRequest	61
ChatResponseData	61
29. DEEP CODE — NotificationWorker Exact Values	61
Notification IDs	61
Weather Thresholds (actual code)	61
Location Fallback	61
HTTP Call in Worker	61
Sunrise/Sunset Parsing	61
Notification Channel	62
PendingIntent Flags	62
30. COMPLETE FILE COUNT AND PROJECT STATISTICS	62
31. SOIL SCIENCE BEHIND THE APP	63
Why These Nutrients Matter	63
ISRIC SoilGrids Data Quality	63
Why Corrections Are Needed	63
32. INTERVIEW / VIVA ADDITIONAL QUESTIONS	63

APPENDIX: KEY SHARED PREFERENCES FILES	65
APPENDIX: ASSETS DIRECTORY	65
APPENDIX: NOTIFICATION ID ALLOCATION TABLE	65

MITTIMITRA ULTIMATE GUIDE

Team A-14, Amrita Vishwa Vidyapetham

B.Tech Project — Amrita School of Computing

TABLE OF CONTENTS

1. Project Identity
 2. What MittiMitra Does — The Big Picture
 3. Technology Stack
 4. Build System — Every Gradle File Explained
 5. AndroidManifest.xml — Full Breakdown
 6. Application Lifecycle — MittiMitraApp.java
 7. All 26 Activities — Function by Function
 8. Database Architecture — Room v8
 9. Network Layer — All 6 API Services
 10. Firebase Services
 11. ML / AI Pipeline
 12. Background Processing — NotificationWorker
 13. Repository Pattern — Data Layer
 14. Security Architecture
 15. Utility Classes
 16. Localization — 14 Languages
 17. Data Flow — End to End (Scan to Recommendation)
 18. Important APIs Used and Why
 19. Deployment — Build, Sign, Release
 20. Function-to-Function Mapping
 21. Important Functions List
 22. Project Nuances and Gotchas
 23. Viva Questions Prep
-

1. PROJECT IDENTITY

App Name: Mitti Mitra (means “Friend of Soil” in Hindi) **Package Name (release):** in.mittimitra.app **Package Name (code namespace):** com.mittimitra **Firebase Project ID:** mitti-mitra-4ee33 **Firebase Project Number:** 512527213662 **Storage Bucket:** mitti-mitra-4ee33.firebaseio.com **Version:** 1.0.5 (versionCode 5) **Min SDK:** 24 (Android 7.0 Nougat) **Target SDK:** 36 (Android 14) **Compile SDK:** 36 **Java Version:** 17 **Guide:** Dr. Swaminathan J

Team Members: Ceija, Rithika Parlapally, Riya Jaiswal, Archana Prabhakaran Nair **Institution:** Amrita Vishwa Vidyapetham, Amrita School of Computing

2. WHAT MITTIMITRA DOES — THE BIG PICTURE

MittiMitra is an intelligent farming assistant for Indian farmers. Here is what it actually does:

Primary Feature — Soil Analysis: User takes a photo of their soil. The app runs a TensorFlow Lite ML model locally on-device to classify soil type (10 types: Alluvial, Black, Clay, Red, Sandy, Loam, Laterite, Yellow, Peaty, Chalky). Simultaneously, it fetches real satellite soil data (nitrogen, pH, organic carbon, clay content) from ISRIC SoilGrids API using GPS coordinates. It applies intelligent corrections based on visual brightness (dark soil = high organic matter = more nitrogen) and soil type heuristics. The combined result feeds into an AI recommendation engine (Firebase Cloud Functions + Groq Llama 3.3 70B) that gives personalized fertilizer and crop advice in the user's language.

Secondary Features: - Plant Doctor: Photo-based plant disease detection via HuggingFace models through backend - Weather Alerts: Real-time agricultural weather from Open-Meteo (temperature, humidity, soil moisture, 7-day forecast) - Mandi Prices: Live commodity prices from data.gov.in Government Open Data API - Crop Calendar: AI-generated crop planting and harvest schedule - Farm Task Planner: Structured task management for farm operations - Government Schemes: PM-KISAN, PM Fasal Bima Yojana, Soil Health Card information - AI Tips (Kisan Sahayak): Voice + text chat with Groq-powered Llama model - Irrigation Calculator: Soil-type-aware water requirement estimation - Documents: Store farm documents (land deeds, certificates) locally with expiry alerts - History: Full scan history with CSV export and comparison - Unit Converter: Agricultural measurements - 14 Languages including Hindi, Tamil, Telugu, Kannada, Marathi, Bengali, Gujarati, Punjabi, and 6 northeast languages

User Types: - Authenticated: Full feature access, cloud sync, profile - Guest: Local only, no cloud sync, profile hidden

3. TECHNOLOGY STACK

Category	Technology	Version	Why
Language	Java (primary)	17	B.Tech project baseline. Kotlin plugin exists but not used in main code.
UI	Android Views + Material Design	Material3	Native Android, no Compose

Category	Technology	Version	Why
Local Database	Room (SQLite wrapper)	Latest stable	Type-safe SQL with migrations
Cloud Database	Firebase Firestore	BOM-managed	Real-time sync, offline support
Authentication	Firebase Auth	BOM-managed	Google, Phone OTP, Email/Password
File Storage	Firebase Storage	BOM-managed	Profile photos
Crash Reporting	Firebase Crashlytics	BOM-managed	Production error tracking
Security	Firebase App Check	BOM-managed	Prevents unauthorized API access
Analytics	Firebase Analytics	BOM-managed	User behavior tracking
Cloud Functions	Firebase Functions	BOM-managed	AI calls from backend (security)
HTTP Client	OkHttp3	Latest	Timeouts, logging interceptor
REST Framework	Retrofit 2	Latest	Type-safe API calls
JSON	Gson	Latest	Serialization/deserialization
ML (on-device)	TensorFlow Lite	Latest	Soil classification, offline
ML (cloud)	Groq Llama 3.3 70B	API	Natural language recommendations
ML (cloud)	HuggingFace via backend	API	Plant disease detection
Images	Glide	4.16.0	Image loading, caching
Image View	CircleImageView	Latest	Profile photos
Background Tasks	WorkManager	2.9.0	Periodic notifications, reminders

Category	Technology	Version	Why
Location	FusedLocationProviderClient	play-services-location	GPS coordinates
Soil API	ISRIC SoilGrids v2.0	REST	Satellite soil property data
Weather API	Open-Meteo	REST	Free, no key, agriculture-specific
Mandi API	data.gov.in (Agmarknet)	REST	Government commodity prices
Web Scraping	jsoup	1.15.4	Scheme data parsing
Loading Animation	Facebook Shimmer	0.5.0	Skeleton loading UI
Preferences	AndroidX Security Crypto	1.1.0-alpha06	Encrypted SharedPreferences
Preferences Store	DataStore	1.0.0	Modern key-value storage
UI Refresh	SwipeRefreshLayout	1.1.0	Pull-to-refresh
CI/CD	GitHub Actions	-	Automated build checks
Obfuscation	ProGuard	Android default	APK size reduction, code protection

4. BUILD SYSTEM — EVERY GRADLE FILE EXPLAINED

4a. Project-Level build.gradle

```

plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.kotlin.compose) apply false
    id 'com.google.gms.google-services' version '4.4.4' apply false
    alias(libs.plugins.firebase.crashlytics) apply false
}

```

What this does: - Declares plugins at project scope. **apply false** means they are available but not applied here — each module applies them separately. - **com.google.gms.google-services** (version 4.4.4) processes google-services.json into BuildConfig constants and wires Firebase. - Kotlin plugin

present because Android Gradle Plugin 8.x requires it even for Java projects (it handles Kotlin compilation internally).

4b. App-Level build.gradle (THE CRITICAL ONE)

File: app/build.gradle

Plugin Section

```
plugins {  
    id 'com.android.application'  
    id 'org.jetbrains.kotlin.android'  
    id 'com.google.gms.google-services'  
    alias(libs.plugins.firebase.crashlytics)  
}
```

Local Properties Loading

```
Properties localProps = new Properties()  
if (rootProject.file('local.properties').exists()) {  
    localProps.load(rootProject.file('local.properties').newDataInputStream())  
}
```

Why: API keys (GROQ, DATA_GOV, OPENWEATHER, BACKEND_BASE_URL) are stored in local.properties which is gitignored. They get injected into BuildConfig at compile time, never hardcoded in source.

Keystore Properties Loading

```
Properties keystoreProps = new Properties()  
def keystorePropsFile = rootProject.file('keystore.properties')  
if (keystorePropsFile.exists()) {  
    keystoreProps.load(keystorePropsFile.newDataInputStream())  
}
```

Why: Release signing keystore path, passwords, alias are in a file that never goes to git. This pattern allows CI to inject signing config as environment variables.

android { } Block Explained

namespace: com.mittimitra — Java package namespace for generated R class and BuildConfig.

compileSdk 36: The API level the code is compiled against. You can use features up to API 36.

applicationId “in.mittimitra.app”: This is what Google Play sees. Different from namespace — allows code refactoring without changing Play Store identity.

minSdk 24: Minimum Android version supported (Android 7.0, Nougat). About 95%+ of active devices qualify.

targetSdk 36: Tells Android OS which behavior compatibility rules to apply. Targeting 36 means modern scoped storage, notification permissions, etc.

versionCode 5: Integer, incremented every release. Play Store uses this to determine if an update is newer.

versionName “1.0.5”: Human-readable version string shown to users.

testInstrumentationRunner: Points to AndroidJUnitRunner for instrumented tests.

BuildConfig Fields (Injected from local.properties)

```
buildConfigField "String", "OPENWEATHER_API_KEY", "..."  
buildConfigField "String", "DATA_GOV_API_KEY", "..."  
buildConfigField "String", "BACKEND_BASE_URL", "..."
```

These become static fields in BuildConfig.java accessible from Java code as BuildConfig.DATA_GOV_API_KEY. They are injected at build time from local.properties so never hardcoded.

Signing Config

```
signingConfigs {  
    release {  
        storeFile file(keystoreProps['storeFile'])  
        storePassword keystoreProps['storePassword']  
        keyAlias keystoreProps['keyAlias']  
        keyPassword keystoreProps['keyPassword']  
    }  
}
```

Why separate: Debug builds auto-sign with Android Studio’s debug keystore. Release builds need the production keystore whose SHA-1 is registered in Firebase and Google Play.

Build Types

debug: - ENABLE_APP_CHECK = false — Firebase App Check disabled in debug so developers can call APIs without Play Integrity token - ENABLE_LOCAL_AI_FALLBACK = true — Groq API enabled as fallback in debug builds - GROQ_API_KEY = actual key from local.properties

release: - signingConfig signingConfigs.release — Uses production keystore - minifyEnabled true — Enables ProGuard code shrinking - shrinkResources true — Removes unused resources (images, strings, layouts) - ENABLE_APP_CHECK = true — Firebase App Check enforced in production via Play Integrity - ENABLE_LOCAL_AI_FALLBACK = false — Groq key empty; all AI goes through backend - proguardFiles — Applies both default Android rules AND our custom rules

compileOptions

Java 17 source and target compatibility. Allows using Java 17 language features.

buildFeatures

- viewBinding true — Generates type-safe binding class per layout (replaces findViewById).
- buildConfig true — Explicitly enables BuildConfig class generation (required in newer AGP).

lint

- `baseline = file("lint-baseline.xml")` — Existing lint issues are snapshotted; new issues fail CI.
- `abortOnError true` — Builds fail on new lint errors.
- `checkReleaseBuilds true` — Lint runs on release builds too.

Dependencies — What Each Library Does

<code>implementation libs.androidx.appcompat</code>	- AppCompatActivity, ActionBar, theme compat
<code>implementation libs.material</code>	- Material Design components (cards, buttons, c
<code>implementation libs.androidx.constraintlayout</code>	- Flexible layout with constraint-based positio
<code>implementation "androidx.security:security-crypto:1.1.0-alpha06"</code>	- EncryptedSharedPreferences
<code>implementation libs.androidx.room.runtime</code>	- Room database runtime
<code>annotationProcessor libs.androidx.room.compiler</code>	- Room annotation processing (generates SQL)
<code>implementation libs.androidx.swiperefreshlayout</code>	- Pull-to-refresh UI
<code>implementation libs.play.services.location</code>	- FusedLocationProvider for GPS
<code>implementation libs.okhttp</code>	- HTTP client with interceptors
<code>implementation libs.logging.interceptor</code>	- Logs all HTTP requests/responses in debug
<code>implementation libs.retrofit</code>	- REST client framework
<code>implementation libs.retrofit.converter.gson</code>	- JSON <-> Java object conversion
<code>implementation libs.gson</code>	- JSON parsing library
<code>implementation platform(libs.firebase.bom)</code>	- Firebase Bill of Materials (locks all Firebase
<code>implementation libs.google.firebase.auth</code>	- Authentication
<code>implementation libs.google.firebase.firestore</code>	- Cloud NoSQL database
<code>implementation libs.google.firebase.storage</code>	- Cloud file storage
<code>implementation libs.firebase.crashlytics</code>	- Crash reporting
<code>implementation libs.firebase.appcheck.playintegrity</code>	- Production app integrity
<code>implementation libs.firebase.appcheck.debug</code>	- Debug app check bypass
<code>implementation "com.google.firebase:firebase-functions"</code>	- Cloud Functions invocation
<code>implementation libs.play.services.auth.v2070</code>	- Google Sign-In
<code>implementation libs.glide.v4160</code>	- Image loading + disk/memory caching
<code>annotationProcessor libs.compiler</code>	- Glide annotation processor
<code>implementation libs.circleimageview</code>	- Circular ImageView for profile photos
<code>implementation libs.firebase.ml.modeldownloader</code>	- Download TFLite models from Firebase
<code>implementation libs.tensorflow.lite</code>	- On-device ML inference engine
<code>implementation 'com.google.guava:guava:31.1-android'</code>	- Collections, async utilities
<code>implementation 'androidx.work:work-runtime:2.9.0'</code>	- WorkManager for background tasks
<code>implementation 'org.jsoup:jsoup:1.15.4'</code>	- HTML parsing for scheme data
<code>implementation 'com.facebook.shimmer:shimmer:0.5.0'</code>	- Shimmer loading animations
<code>implementation 'androidx.datastore:datastore-preferences:1.0.0'</code>	- Modern preferences storage
<code>implementation 'com.google.firebase:firebase-analytics'</code>	- Usage analytics

4c. settings.gradle

```
dependencyResolutionManagement {  
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)  
    repositories {  
        google()  
    }  
}
```

```

        mavenCentral()
    }
}
rootProject.name = "MittiMitra"
include ':app'

```

What this does: Declares where Gradle downloads libraries from. `FAIL_ON_PROJECT_REPOS` means repos can only be declared here (not in individual build.gradle files) — enforced in modern Gradle. Single module project: only `:app`.

4d. gradle.properties

```

android.useAndroidX=true
android.enableJetifier=false (implied)
kotlin.code.style=official
android.nonTransitiveRClass=true
org.gradle.jvmargs=-Xmx2048m

```

android.nonTransitiveRClass=true: Each module only sees its own R class, not transitive dependencies' R. Faster builds, smaller DEX. **org.gradle.jvmargs=-Xmx2048m:** Gradle daemon gets 2GB RAM. Prevents OOM during large builds.

4e. proguard-rules.pro — Full Rules Explained

ProGuard runs during release builds. It: 1. **Shrinks:** Removes unused classes, methods, fields 2. **Optimizes:** Inlines methods, removes dead branches 3. **Obfuscates:** Renames classes/methods to a, b, c etc.

Our custom rules preserve:

```
-keepattributes SourceFile,LineNumberTable
```

Keeps crash stack traces readable in Crashlytics.

```
-keepattributes Signature,InnerClasses,EnclosingMethod
```

Required for Gson generics and Retrofit type inference.

```

# Retrofit
-keep interface com.mittimitra.network.*
-keepclassmembers class * extends retrofit2.Callback { *; }

```

Retrofit uses reflection to call interface methods. Must not be renamed.

```

# Room
-keep @androidx.room.* class *
-keepclassmembers @androidx.room.* class * { *; }

```

Room generates code that references entity field names. Renaming breaks SQL column mapping.

```

# Gson models
-keep class com.mittimitra.backend.model.** { *; }
-keep class com.mittimitra.network.** { *; }
-keep class com.mittimitra.database.entity.** { *; }

```

Gson uses reflection to map JSON keys to field names. If fields are renamed, JSON parsing breaks.

```
# TensorFlow Lite
-keep class org.tensorflow.lite.** { *; }
```

TFLite native bridge uses reflection.

```
# WorkManager
-keep class * extends androidx.work.Worker
-keep class * extends androidx.work.ListenableWorker
```

WorkManager instantiates workers by class name at runtime.

```
# jsoup
-keep class org.jsoup.** { *; }
```

Used for HTML parsing of scheme data.

5. ANDROIDMANIFEST.XML — FULL BREAKDOWN

File: app/src/main/AndroidManifest.xml

Permissions Declared

Permission	Why
INTERNET	All network calls (APIs, Firebase)
ACCESS_NETWORK_STATE	Check if online before API calls
CAMERA	Soil photo capture, plant scan
ACCESS_FINE_LOCATION	GPS coordinates for soil/weather data
ACCESS_COARSE_LOCATION	Fallback location when GPS unavailable
READ_EXTERNAL_STORAGE (maxSdkVersion=32)	Gallery access on Android 12 and below
READ_MEDIA_IMAGES	Gallery access on Android 13+ (new permission split)
POST_NOTIFICATIONS	Required on Android 13+ for showing notifications
RECORD_AUDIO	Voice input in Kisan Sahayak / TipActivity

Why two storage permissions: Android 13 (API 33) replaced `READ_EXTERNAL_STORAGE` with granular `READ_MEDIA_IMAGES`, `READ_MEDIA_VIDEO`, etc. We declare both with `maxSdkVersion` to handle both old and new Android.

Application Attributes

`android:name=".MittiMitraApp"` - Custom Application class
`android:allowBackup="true"` - System backup allowed
`android:icon="@drawable/mittimitra_logo"`
`android:roundIcon="@drawable/mittimitra_logo"`
`android:supportsRtl="true"` - Right-to-left language support
`android:theme="@style/Theme.MittiMitra"`

Activity List (26 Activities)

SplashActivity — Launcher (`android:exported="true"`) - Only exported activity (entry point from home screen/deep links) - Intent filter: MAIN + LAUNCHER - Intent filter: `mittimitra://` deep link scheme

All other activities have `android:exported="false"` — cannot be launched by external apps (security).

Authentication Flow: - LoginActivity -> SignupActivity -> WelcomeActivity - ForgotPasswordActivity (standalone reset)

Main App: - HomeActivity -> ScanActivity -> RecommendationActivity - HistoryActivity, DocumentsActivity, TipActivity (chat) - ContactActivity, ProfileActivity

Settings Group: - SettingsActivity -> LanguageActivity, AccessibilityActivity, ThemeSettingsActivity - ManageDataActivity, AccountSecurityActivity, LinkEmailActivity, HelpActivity - NotificationSettingsActivity

Tools: - UnitConverterActivity, CompareActivity - PlantScanActivity, FarmTaskPlannerActivity

New Feature Cards: - SchemeActivity, WeatherAlertsActivity, CropCalendarActivity - IrrigationActivity, MandiActivity

FileProvider

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.provider"
    android:grantUriPermissions="true"
    android:exported="false">
    <meta-data android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

Why: Android 7.0+ forbids sharing raw `file://` URIs between apps (camera intent needs a URI for where to save full-resolution photo). FileProvider creates a `content://` URI that grants temporary read permission to the camera app. `${applicationId}` resolves to “`in.mittimitra.app`” in release, “`com.mittimitra`” in debug — avoiding authority conflicts.

file_paths.xml defines: `camera_images` path inside `getCacheDir()` — this is where ScanActivity and PlantScanActivity create temp JPEG files for camera capture.

6. APPLICATION LIFECYCLE — MittiMitraApp.java

File: app/src/main/java/com/mittimitra/MittiMitraApp.java **Extends:** Application **Purpose:** Runs before any Activity. Initializes all global singletons.

Step 1: `FirebaseApp.initializeApp(this)`

Must be first. All Firebase services depend on this. If skipped, any Firebase call crashes.

Step 2: `AnalyticsHelper.init(this)`

Initializes Firebase Analytics. Logs user journey events throughout the app.

Step 3: `Firebase App Check (conditional)`

If `BuildConfig.ENABLE_APP_CHECK` is true: - Debug: `DebugAppCheckProviderFactory` — generates a debug token (printed to Logcat), registered in Firebase Console for dev testing - Release: `PlayIntegrityAppCheckProviderFactory` — uses Google Play Integrity API to verify the app is genuine, not modified, running on real hardware

Without App Check, the Firebase backend would accept calls from any app or curl command using the project config. App Check ensures only the real MittiMitra app can access the backend.

Step 4: `Firestore offline persistence`

Enables Firestore to cache documents locally so the app works offline. Uses reflection-based forward compatibility: - If new `PersistentCacheSettings` API exists (Firestore SDK $\geq 24.x$): uses it - Falls back to deprecated `setPersistenceEnabled(true)` for older SDKs

Step 5: `WorkManager scheduling`

```
PeriodicWorkRequest notifRequest = new PeriodicWorkRequest.Builder(
    NotificationWorker.class, 15, TimeUnit.MINUTES).build();
```

```
WorkManager.getInstance(this).enqueueUniquePeriodicWork(
    "DailyAlerts",
    ExistingPeriodicWorkPolicy.KEEP,
    notifRequest);
```

Schedules `NotificationWorker` to run every 15 minutes. - `enqueueUniquePeriodicWork` with `KEEP` policy: if “DailyAlerts” already scheduled, do nothing. Prevents duplicate scheduling on app restarts. - Called here (not in `HomeActivity`) so it’s set up once per app install, not every time `HomeActivity` opens.

7. ALL 26 ACTIVITIES — FUNCTION BY FUNCTION

BaseActivity.java

All activities extend `BaseActivity`. Handles: - Theme application (light/dark/system) from `AppPreferences` - Font scale (small/normal/large/dyslexic) application - Language locale override (forces selected language for all activities) - `Objects.equals()` null-safe comparison for theme strings

Key method: `attachBaseContext(Context newBase)` — Overrides locale before activity creates. This is how language switching works per-activity.

SplashActivity.java

Purpose: App entry point. Shows logo, checks auth state, routes user.

Flow: 1. Show splash screen (logo + animation) 2. Check Firebase Auth: `FirebaseAuth.getInstance().getCurrentUser()` 3. If logged in -> `HomeActivity` 4. If not -> `LoginActivity`

Timing: 1.5-2 second delay for branding, then route.

LoginActivity.java

Purpose: Authentication hub. Three methods: Google, Phone OTP, Email.

Key functions:

`setupGoogleSignIn()` - Creates `GoogleSignInClient` with `GoogleSignInOptions.Builder(DEFAULT_SIGN_IN).setWebClientId(WEB_CLIENT_ID)` - `WEB_CLIENT_ID` comes from `google-services.json` (not the Android client ID — the web client ID) - `requestIdToken` is mandatory for Firebase auth

`handleGoogleSignInResult(Intent data)` - Gets `GoogleSignInAccount` from intent - Extracts ID token - Calls `firebaseAuthWithGoogle(idToken)`

`firebaseAuthWithGoogle(String idToken)` - Creates `GoogleAuthProvider.getCredential(idToken, null)` - Calls `auth.signInWithCredential(credential)` - On success: checks if this Google email was previously registered via phone OTP (duplicate account scenario) - If duplicate found in Firestore -> launches dialog, deletes orphan Firebase Auth account, tries to auto-link phone

`sendOtp(String phoneNumber)` - Validates 10-digit phone - Adds +91 country code - Calls `PhoneAuthProvider.verifyPhoneNumber()` with Firebase - Shows OTP input section on success - 60-second timeout

`verifyOtp(String code)` - Creates `PhoneAuthProvider.getCredential(verificationId, code)` - Calls `auth.signInWithCredential()` - On success: saves session, routes to `HomeActivity` or `WelcomeActivity`

`loginWithEmail()` - `auth.signInWithEmailAndPassword(email, password)`

`applyPendingPhoneLink()` - After Google sign-in: checks `SharedPreferences` for a saved pending phone number from a previous orphan account resolution - Saves that phone to Firestore `farmer` document so the account is properly linked

Duplicate Account Nuance: A user who signed up with phone OTP later signs in with Google using the same email. Firebase creates two separate accounts. The app detects this: it finds the phone number in the Google account, checks Firestore for a `farmer` document with that phone, and if found, deletes the new (empty) Google Auth account and forces the user to sign in with the original phone OTP method. On re-sign-in, `applyPendingPhoneLink()` stores the phone in the Firestore document.

SignupActivity.java

Purpose: New user registration.

Validates: - Name: not empty - Email: `Android Patterns.EMAIL_ADDRESS` regex - Phone: 10-digit numeric

Flow: 1. `auth.createUserWithEmailAndPassword(email, password)` 2. On success: create Firestore document at `farmers/{uid}` with all fields 3. Save session via `SessionManager` 4. Route to `HomeActivity`

Firestore document structure on create:

```
{
    firstName, phone, email, aadhaarId, kisanId,
    dob, createdAt, landSize, cropName, district,
    state, village, profileImageUrl
}
```

These exact field names are enforced by Firestore security rules `hasOnly([...])`.

WelcomeActivity.java

Purpose: Onboarding screen for new users. Shown after first successful registration.

- `ViewPager2` with onboarding slides
 - Skippable (goes to `HomeActivity`)
-

ForgotPasswordActivity.java

Purpose: Password reset via email.

Key functions:

`resetPassword()` - Validates email format - Disables button during async to prevent double-tap - Calls `auth.sendPasswordResetEmail(email)` - Handles `FirebaseAuthInvalidUserException` (email not found) vs other errors separately - Success: shows confirmation, re-enables button

HomeActivity.java

Purpose: Main dashboard. Navigation hub.

Key functions:

`setupCarousel()` - `ViewPager2` with 3 scheme banners - Images: `R.drawable.banner_pm_kisan`, `banner_soil_health`, `banner_fasal_bima` - Titles from `strings.xml` (so language switching works) - `CarouselAdapter` wraps images + titles

`carouselRunnable` - Handler-based auto-scroll every 3000ms - Loops through carousel items with modulo - Stops on `onPause()`, resumes on `onResume()`

setupGridNavigation() - 13 cards mapped to activities: - `card_scan` -> `ScanActivity` - `card_history` -> `HistoryActivity` - `card_documents` -> `DocumentsActivity` - `card_tips` -> `TipActivity` - `card_recommendation` -> `RecommendationActivity` - `card_help` -> `HelpActivity` - `card_plant_doctor` -> `PlantScanActivity` - `card_scheme_promo` -> `SchemeActivity` - `card_weather` -> `WeatherAlertsActivity` - `card_calendar` -> `CropCalendarActivity` - `card_irrigation` -> `IrrigationActivity` - `card_mandi` -> `MandiActivity` - `card_farm_tasks` -> `FarmTaskPlannerActivity`

setupBottomNavigation() - `nav_scan` -> `ScanActivity` - `nav_profile`: hidden for guests, shows `ProfileActivity` for authenticated users - `nav_settings` -> `SettingsActivity`

refreshUserNameFromFirestore() - On every `onResume()`, queries `farmers/{uid}` for `firstName` - Updates welcome message text and `SessionManager` cache - Why: if user updates their name in `ProfileActivity`, `HomeActivity` shows it instantly on back

onBackPressed() - Double-press-to-exit pattern - First press: shows “Press back again to exit” toast, records timestamp - Second press within 2000ms: calls `finishAffinity()` to close entire app stack

ScanActivity.java

Purpose: Core feature. Soil analysis via camera + ML + satellite data.

Key member variables: - `analysisExecutor` — Single-thread `ExecutorService`. All background work (geocoding, TFLite inference, report generation) runs here. Never on main thread. - `cameraImageUri` — Full-resolution URI created via `FileProvider` for camera capture - `currentImageBitmap` — In-memory bitmap of captured/selected image - `sensorManager`, `lightSensor` — Android hardware sensor for ambient light - `isLowLight` — State flag; prevents repeated snackbar spam - `finalN`, `finalP`, `finalK`, `finalpH` — Nutrient values, populated from `SoilGrids API`

SOIL_LABELS array (10 types): "Alluvial", "Black", "Clay", "Red", "Sandy", "Loam", "Laterite", "Yellow", "Peaty", "Chalky" This order must exactly match the TFLite model's output layer order.

Launchers (ActivityResultContracts):

cameraLauncher (`TakePicture contract`) - Returns boolean success - On success: `decodeSampledBitmap(cameraImage, 1024)` — decodes full-res photo scaled down to max 1024px to avoid OOM

galleryLauncher (`PickVisualMedia contract`) - Returns `content:// URI` - `decodeSampledBitmap(result, 1024)` same decode pipeline

requestPermissionLauncher - Handles `CAMERA` runtime permission - On grant: launches camera - On deny: shows error toast

Key functions:

checkLocationAndFetchData() - Checks `ACCESS_FINE_LOCATION` permission - `fusedLocationClient.getLastLocation()` — gets last known GPS fix (fast, no fresh request) - On success: 1. Caches lat/lon to `user_location_cache` `SharedPreferences` (used by `NotificationWorker`) 2. Calls `fetchAddress()`, `fetchAgroData()`, `fetchLiveSoilData()` in parallel

`fetchAddress(double lat, double lon)` - Runs on `analysisExecutor` (geocoding is slow/blocking)
- Android Geocoder — converts GPS to human-readable address - Extracts: locality (city), subAdminArea (district), adminArea (state) - Posts result to main thread via Handler

`fetchAgroData(double lat, double lon)` - Calls Open-Meteo via Retrofit (background thread via Retrofit's OkHttp dispatcher) - Parses: temperature, humidity, precipitation, wind_speed, weather_code, soil_moisture_0_to_1cm - `WeatherUtils.getWeatherDescription(weatherCode)` -> emoji + description string - `WeatherUtils.getAgriculturalRecommendations()` -> actionable alerts (high humidity = fungal risk, etc.) - Shows Snackbar if alert is not "[OK]" (success) - Caches to `scan_cache` SharedPreferences

`fetchLiveSoilData(double lat, double lon)` - Calls ISRIC SoilGrids API: - Properties requested: ["nitrogen", "phh2o", "soc", "clay"] - Depth: "0-5cm", value type: "mean" - Parses JSON: `properties.layers` is a `JSONArray`, loops through each layer - Unit conversions: - nitrogen: cg/kg -> multiply by 2.0 to get approximate kg/ha - phh2o: pH*10 -> divide by 10 to get real pH - soc (soil organic carbon): $P = 10 + (soc * 0.2)$ heuristic - clay: $K = 50 + (clay\% * 3.5)$ heuristic

`startAnalysis()` - Guard: image must be selected - Disables button, shows progress bar - Dispatches `runLocalInference()` to `analysisExecutor`

`runLocalInference(Bitmap bitmap, String userNotes)` - Loads TFLite model: `loadModelFile()` via `MappedByteBuffer` (memory-mapped file, zero-copy) - Creates `Interpreter` with loaded model - Resizes bitmap to 224x224 (model input size) - Creates `ByteBuffer`: 4 bytes $\times$ 224 $\times$ 224 $\times$ 3 channels = ~600KB - Fills buffer: pixel RGB values normalized to 0.0–1.0 float - `interpreter.run(inputBuffer, output)` — output is `float[1][10]` - `getMaxIndex(output[0])` — argmax for classification - Maps index to `SOIL_LABELS` string - Calls `generateSmartReport(detectedSoil, userNotes)`

`generateSmartReport(String detectedSoilType, String userNotes)` - Runs on `analysisExecutor` (has Room write) - `applySmartCorrections(detectedSoilType)` — adjusts N, P, K using heuristics - Builds `JSONObject`: {N, P, K, pH, weather, soil_dynamic, location, detected_soil, user_notes} - Creates `SoilAnalysis` entity, inserts into Room DB - Gets `userId` via `UserIdentityResolver.getActiveUserId()` - Saves last soil type to `AppPreferences` (used by `IrrigationActivity`) - Posts to main thread: starts `RecommendationActivity` with `DETECTED_SOIL_TYPE` extra, finishes self

`applySmartCorrections(String detectedSoilType)` - Intelligence layer: adjusts satellite N/P/K values using visual evidence - Per soil type heuristics (based on Indian soil science): - Black soil (Regur): $N \times 1.25$, $K \times 1.2$ (retains nutrients well) - Red soil: $N \times 0.9$, $P \times 0.8$ (iron fixation reduces available P) - Sandy: $N/P/K \times 0.7-0.8$ (high leaching) - Clay: $K \times 1.15$ (high K retention) - Laterite: $N \times 0.8$, $P \times 0.7$, $K \times 0.7$ (acidic, leached) - Alluvial: $N \times 1.1$, $K \times 1.1$ (generally fertile) - Visual brightness from `calculateBrightness()`: - Dark soil (brightness < 100): $N += 15\%$ (dark = high organic matter = high N) - Very light soil (brightness > 180): $N -= 15\%$

`calculateBrightness(Bitmap bitmap)` - Samples center 50x50 pixels of image (relevant soil area, not edges) - Averages RGB channels - Returns 0-255 luminance value

`decodeSampledBitmap(Uri uri, int maxDim)` - Two-pass decode strategy to avoid `OutOfMemoryError`: - Pass 1: `inJustDecodeBounds = true` — reads dimensions without loading pixels - Calculates `inSampleSize` (power of 2) to keep largest dimension $\leq$ maxDim - Pass 2: actual decode with calculated sample size - Uses try-with-resources on `InputStream` (resource leak prevention)

`loadModelFile()` - Opens TFLite model from assets as `AssetFileDescriptor` - Maps to

MappedByteBuffer via FileChannel — efficient memory mapping - Model stays in RAM as long as Interpreter exists

Sensor lifecycle: - `onResume()`: register light sensor listener - `onPause()`: unregister to save battery - `onSensorChanged()`: if lux < 50 and not already warned -> show “Too dark” Snackbar

Thread safety: - `analysisExecutor = Executors.newSingleThreadExecutor()` — single thread, operations serialized - All UI updates wrapped in `runOnUiThread()` - Executor shut down in `onDestroy()` to prevent memory leaks

RecommendationActivity.java

Purpose: Displays soil analysis results, health score, AI advisory, PDF export.

Key functions:

`loadLatestReport()` - Queries Room DB: `soilDao().getLatestReportForUser(userId)` - Runs on `dbExecutor` - Parses stored JSON: extracts N, P, K, pH, detected_soil, location, weather - Posts to main thread to populate UI

`calculateSoilHealthScore(double pH)` - Base: 100 points - Deduct 15 points per 1.0 pH unit deviation from 7.0 (neutral) - pH 6.0 -> score 85, pH 5.0 -> score 70, pH 9.0 -> score 70 - Score buckets: - >= 80: “Excellent Condition” (green) - 60-79: “Good Condition” (amber) - 40-59: “Needs Attention” (orange) - < 40: “Poor Condition” (red)

`fetchAdvisory(JsonObject reportJson)` - Calls `FirestorePredictionRepository.fetchSoilAdvisory()` - Backend receives: N, P, K, pH, soil_type, location, weather, crop - Returns: markdown text with fertilizer recommendations

`downloadPdf()` - Creates PDF via `PdfDocument` API - Writes nutrient table, health score, AI advisory text - Saves to Downloads folder (or internal files for API < 29) - Uses try-with-resources for `OutputStream`

`shareReport()` - Shares PDF file via Android share sheet - `FileProvider` URI for secure file sharing - Validates PDF exists first (error if not saved yet)

`speakAdvisory()` - `TextToSpeech` engine narrates advisory text - Language set to user’s selected locale

`suggestTasks(SoilAdvisoryData advisory)` - Calls `TaskSuggestionEngine.suggestFromSoilAdvisory(advisory, userId)` - Runs on `dbExecutor` (Room writes) - Creates `FarmTask` entries for nutrient corrections

HistoryActivity.java

Purpose: View all past soil scans, export to CSV, compare analyses.

Key functions:

`loadHistory()` - Queries Room: `soilDao().getAnalysisForUser(userId)` - Runs on `dbExecutor` (`ExecutorService`, not raw `Thread`) - Populates `RecyclerView` via `RecentAnalysisAdapter`

`exportCsv()` - Gets all analyses for user - Writes CSV: date, detected_soil, N, P, K, pH - JSON key mapping: "N"/"P"/"K"/"pH" (matches ScanActivity output) - FileWriter in try-with-resources - Shares file via Intent.ACTION_SEND

`onItemClick(SoilAnalysis analysis)` - Starts CompareActivity with selected analysis ID

CompareActivity.java

Purpose: Side-by-side comparison of two soil analyses.

Key functions:

`loadReports()` - Queries Room for both analyses by ID - Parses JSON with correct keys ("N"/"P"/"K"/"pH" — not full names) - Displays delta indicators (Δ) between the two readings

Nuance: Early version used wrong JSON keys ("nitrogen"/"phosphorus"/"potassium"/"ph") which didn't match ScanActivity's JSONObject output. Fixed to match exact keys.

TipActivity.java (Kisan Sahayak)

Purpose: AI chat interface. Text + voice input.

Key functions:

`sendMessage(String text)` - Calls `FirestorePredictionRepository.fetchChatResponse()` - Backend: Groq Llama 3.3 70B with farmer context - Displays response in RecyclerView via ChatAdapter - Saves to Room chat_messages table

`startVoiceRecognition()` - `SpeechRecognizer` via `RecognizerIntent` - Requires RECORD_AUDIO permission - Transcript -> `sendMessage()`

`loadOfflineTips(String query)` - Falls back to local `offline_tips.json` in assets - InputStream in try-with-resources - Parsed and displayed if backend unavailable

`speakResponse(String text)` - TextToSpeech narration of AI response

ProfileActivity.java

Purpose: View/edit user profile. Unique phone enforcement.

Key functions:

`loadProfile()` - Queries Firestore: `farmers/{uid}` - Populates: name, email, phone, aadhaar, district, state, village

`updateProfile()` - Validates fields (name, email format, aadhaar 12 digits) - Calls `checkPhoneUniqueness()` before save

`checkPhoneUniqueness()` - Queries Firestore for other farmers with same phone number - On PERMISSION_DENIED error: falls through to `doFirestoreSave()` (phone uniqueness via Cloud

Function, not client-side query — Firestore rules deny client listing) - Blocks save if duplicate found, shows error

`doFirestoreSave()` - Updates Firestore document with profile data - Updates SessionManager with new name

`uploadProfilePhoto(Uri imageUri)` - Compresses image via `ImageCompressor` - Uploads to Firebase Storage: `profile_images/{uid}/profile.jpg` - Gets download URL -> saves to Firestore
`profileImageUrl` - Loads via Glide in ProfileActivity

AccountSecurityActivity.java

Purpose: View linked auth providers (Google, Phone, Email). Link/unlink.

Key functions:

`loadProviders()` - `FirebaseAuth.getInstance().getCurrentUser().getProviderData()` - Lists all linked providers

`fetchPhoneFromFirestore()` - When no phone provider in Firebase Auth: fetches from Firestore `farmers/{uid}` phone field - Shows “OK Phone Saved in Profile” if found

`linkEmail()` / `linkPhone()` - Routes to LinkEmailActivity for email linking - Shows coming-soon toast for other providers

DocumentsActivity.java

Purpose: Store and manage farm documents (land deeds, certificates).

Key functions:

`setupRecyclerView()` - Null guard on `documentList` - Must set `LinearLayoutManager` BEFORE `setAdapter()` (crash fix)

`loadDocuments()` - Room: `documentDao().getDocumentsForUser(userId)` - Runs on `ExecutorService`

`pickDocument()` - `ACTION_OPEN_DOCUMENT` intent for any file type - Saves copy to internal storage (not relying on external URI which can be revoked) - Buffer 4096 bytes (not 1024) for performance - `file.delete()` on failure with warning log

`checkExpiringDocuments()` - Queries: documents expiring in next 7 days - Shows notification via `NotificationWorker`

SettingsActivity.java

Purpose: Settings hub with RecyclerView of options.

Key functions:

`setupRecyclerView()` - Must set `LinearLayoutManager` BEFORE `setAdapter()` (identical crash pattern as `DocumentsActivity`) - `SettingsAdapter` wraps `List`

Routes to: `LanguageActivity`, `AccessibilityActivity`, `ThemeSettingsActivity`, `NotificationSettingsActivity`, `ManageDataActivity`, `AccountSecurityActivity`, `HelpActivity`, `ContactActivity`.

LanguageActivity.java

Purpose: Select app language from 14 options.

On selection: - Saves language code to `AppPreferences` - Recreates activity stack via `recreate()` - `BaseActivity.attachBaseContext()` applies locale to all subsequent activities

ThemeSettingsActivity.java

Purpose: Select light/dark/system theme.

`applyTheme(String theme)` - "light": `AppCompatActivity.MODE_NIGHT_NO` - "dark": `AppCompatActivity.MODE_NIGHT_YES` - "system": `AppCompatActivity.MODE_NIGHT_FOLLOW_SYSTEM`

`checkSaved.equals(currentTheme)` -> fixed to `"value".equals(currentSaved)` null-safe Yoda style.

NestedScrollView nuance: Two `CardView` children needed a `LinearLayout` wrapper inside `NestedScrollView` (`NestedScrollView` only accepts one direct child).

ManageDataActivity.java

Purpose: Clear scan history, chat, documents. Manage storage.

Key functions:

`listFiles()` - Null-checks `getCacheDir().listFiles()` before iteration (returns null if empty)

`clearAll()` - Room: clears all user-scoped data (calls `clearHistoryForUser`, `clearDocumentsForUser`, `clearMessagesForUser`) - Runs on `databaseExecutor`, shut down in `onDestroy()`

NotificationSettingsActivity.java

Purpose: Toggle notification categories (weather, mandi, reminders, expiry).

Saves preferences to `MittiMitra_Notifications` `SharedPreferences`. `NotificationWorker` reads these flags before sending each category.

PlantScanActivity.java

Purpose: Plant disease detection via photo + HuggingFace backend.

Key functions:

`analyzeImage(Bitmap bitmap)` - Runs on `ExecutorService` (not raw `Thread`) - Compresses image to Base64 - Calls `FirestorePredictionRepository.fetchPlantDiagnosis()` - Backend: HuggingFace vision model via Firebase Cloud Function - Saves result to Room `plant_health` table - Creates follow-up tasks via `TaskSuggestionEngine.suggestFromPlantDiagnosis()`

`launchCamera()` / `pickGallery()` - Same pattern as `ScanActivity`: `TakePicture` contract + `FileProvider` URI

`onDestroy()` - Calls `executor.shutdown()` (explicit cleanup)

WeatherAlertsActivity.java

Purpose: 7-day weather forecast with agricultural alerts.

Key functions:

`fetchWeather()` - Open-Meteo `get7DayForecast()` API call - Parses daily array: max/min temp, precipitation, weather code per day - Displays in `RecyclerView` via forecast adapter

`generateAlerts()` - Uses `WeatherUtils.getAgriculturalRecommendations()` for actionable advice
- Shows `Snackbar` or card per alert

CropCalendarActivity.java

Purpose: AI-generated crop planting calendar.

Key functions:

`generateCalendar(String cropName)` - Calls `FirestorePredictionRepository.fetchCropSchedule()`
- Backend: Groq Llama generates planting stages, dates, activities - Saves to Room `crop_schedules` table - Creates farm tasks via `TaskSuggestionEngine.suggestFromCropSchedule()` - Runs on `ExecutorService` (Room writes)

`errorBody` closed with `try-with-resources` in error handling.

IrrigationActivity.java

Purpose: Calculate irrigation water requirement based on soil type, crop, weather.

Key functions:

`calculate()` - Retrieves last detected soil type from `AppPreferences` - Gets last weather data (temperature, humidity) - Applies soil-type irrigation multipliers (sandy = more water, clay = less) - Displays result in mm/day or L/hectare

`onFailure()` callbacks -> `Log.e()` (not silent)

MandiActivity.java

Purpose: Live commodity market prices.

Key functions:

`fetchPrices(String commodity, String state)` - data.gov.in Agmarknet API - Resource ID: 9ef84268-d588-465a-a308-a864a43d0070 - Params: api-key, format=json, filters[state], filters[commodity], limit=10 - Displays commodity, market name, min/max/modal price in RecyclerView

`cacheResults()` - Stores JSON to `mandi_price_cache` SharedPreferences - NotificationWorker reads this cache for price notifications

`onFailure` -> `Log.e()` + hardcoded fallback string -> `R.string.mandi_fallback_msg`

SchemeActivity.java

Purpose: Government agricultural schemes browser.

Key functions:

`loadSchemes()` - Reads from `schemes_data.json` in assets (offline-first) - Falls back to Firestore `schemes` collection - Parses via `SchemeRepository` (jsoup for any HTML content)

`openSchemeLink(Scheme scheme)` - If no URL: shows “No link available” toast (`R.string.scheme_no_link`)
- If URL: opens in Chrome Custom Tab or browser

FarmTaskPlannerActivity.java

Purpose: Farm task management. Create, view, complete tasks.

Key functions:

`loadTasks()` - `RoomTaskRepository.getTasksForUser(userId)` - Displays via `FarmTaskAdapter` with priority coloring

`showAddTaskDialog()` - Bottom sheet dialog: title, description, crop, due date, priority - Creates `FarmTask` entity via `RoomTaskRepository.createTask()` - Auto-creates `TaskReminder` entry

`completeTask(long taskId)` - `RoomTaskRepository.completeTask(taskId, now)` - Logs to `task_logs` via repository

UnitConverterActivity.java

Purpose: Agricultural unit conversions.

Validates: non-empty input, numeric format (shows R.string error strings, not hardcoded text).

ContactActivity.java

Purpose: Helpline contacts.

All contact details (phone, email) in R.string.* — not hardcoded in Java. Intent: ACTION_DIAL for phone, ACTION_SENDTO for email, chooser title from R.string.

HelpActivity.java

Purpose: FAQ and help content. Static content display.

LinkEmailActivity.java

Purpose: Link email/password to existing auth account.

```
linkEmail(String email, String password) - EmailAuthProvider.getCredential(email,
password) - user.linkWithCredential(emailCred)
```

AccessibilityActivity.java

Purpose: Font size and dyslexic-friendly font settings.

Saves to AppPreferences. BaseActivity reads and applies via getResources().updateConfiguration().

8. DATABASE ARCHITECTURE — ROOM v8

Overview

Room is an abstraction layer over SQLite. Provides: - Compile-time SQL verification (typos caught at build, not runtime) - Type-safe queries (no raw cursor management) - Migration support (schema evolution without data loss)

Database Class: MittiMitraDatabase.java

```
@Database(entities = {SoilAnalysis.class, Document.class, ChatMessage.class,
                        CropSchedule.class, PlantHealth.class, Field.class,
                        FarmTask.class, TaskReminder.class, TaskLog.class},
version = 8, exportSchema = false)
```

exportSchema = false: No JSON schema export. Trade-off: no migration validation tooling, but no extra file management.

Singleton pattern:

```
private static volatile MittiMitraDatabase INSTANCE;
public static MittiMitraDatabase getDatabase(final Context context) {
    if (INSTANCE == null) {
        synchronized (MittiMitraDatabase.class) {
            if (INSTANCE == null) {
                INSTANCE = Room.databaseBuilder(...).build();
            }
        }
    }
    return INSTANCE;
}
```

Double-checked locking. `volatile` prevents instruction reordering. Thread-safe lazy init.

File: `mitti_mitra_database` (SQLite file in app's private data directory)

Migration History

v1 -> v2: Removed old foreign key constraints from early schema design. Done as a destructive migration in early builds (fallbackToDestructiveMigration removed later).

v2 -> v3: Added `chat_messages` table.

```
CREATE TABLE IF NOT EXISTS chat_messages (
    message_id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    content TEXT, is_user INTEGER NOT NULL, timestamp INTEGER NOT NULL)
```

v3 -> v4: Added `expiry_date` column to documents.

```
ALTER TABLE documents ADD COLUMN expiry_date INTEGER
```

v4 -> v5: Added `user_id` column to `soil_history`, `documents`, `chat_messages`. Multi-user support. Before this, all data was shared across users on same device.

v5 -> v6: Added `crop_schedules` table.

v6 -> v7: Added `plant_health` table.

v7 -> v8: Added `fields`, `farm_tasks`, `task_reminders`, `task_logs` tables (Farm Planner feature).

Entities (9 tables)

SoilAnalysis — `soil_history` | Column | Type | Notes | |———|———|———| | `analysisId` | INTEGER PK AUTOINCREMENT | | `timestamp` | INTEGER | Unix millis | | `soilReportJson` | TEXT | Full JSON from ScanActivity | | `userId` | TEXT | Firebase UID or guest ID |

Document — `documents` | Column | Type | Notes | |———|———|———| | `documentId` | INTEGER PK AUTOINCREMENT | | `documentName` | TEXT | | | `internalFilePath` | TEXT | Path in app internal storage | | `documentType` | TEXT | “Land Deed”, “Certificate”, etc. | | `expiryDate` | INTEGER | Unix millis, nullable | | `userId` | TEXT | |

ChatMessage — `chat_messages` | Column | Type | Notes | |——|——|——-| | messageId | INTEGER PK AUTOINCREMENT | | content | TEXT | Message text | | isUser | INTEGER | Boolean (1=user, 0=AI) | | timestamp | INTEGER | | | userId | TEXT | |

CropSchedule — `crop_schedules` | Column | Type | Notes | |——|——|——-| | id | INTEGER PK AUTOINCREMENT | | userId | TEXT | | | cropName | TEXT | | | plantingDate | TEXT | | | harvestDate | TEXT | | | fullJson | TEXT | Complete AI response JSON | | timestamp | INTEGER | |

PlantHealth — `plant_health` | Column | Type | Notes | |——|——|——-| | id | INTEGER PK AUTOINCREMENT | | | userId | TEXT | | | imagePath | TEXT | Local file path | | cropName | TEXT | | | healthStatus | TEXT | “Healthy”, “Diseased”, etc. | | diagnosis | TEXT | Disease name | | confidence | INTEGER | 0-100 | | fullJson | TEXT | | | timestamp | INTEGER | |

Field — `fields` | Column | Type | Notes | |——|——|——-| | id | INTEGER PK AUTOINCREMENT | | | userId | TEXT | | | fieldName | TEXT | | | cropName | TEXT | | | areaHectares | REAL | | | locationLabel | TEXT | | | notes | TEXT | | | createdAt | INTEGER | | | updatedAt | INTEGER | |

FarmTask — `farm_tasks` | Column | Type | Notes | |——|——|——-| | id | INTEGER PK AUTOINCREMENT | | | userId | TEXT | | | fieldId | INTEGER | FK to fields (not enforced at DB level) | | title | TEXT | | | description | TEXT | | | cropName | TEXT | | | stage | TEXT | “planting”, “growing”, “harvest” | | source | TEXT | “manual”, “ai_soil”, “ai_plant”, “ai_crop” | | dueAt | INTEGER | Unix millis | | status | TEXT | “pending”, “completed” | | priority | INTEGER | 1=low, 2=medium, 3=high | | confidence | INTEGER | AI confidence 0-100 | | metadataJson | TEXT | Additional data | | createdAt | INTEGER | | | completedAt | INTEGER | Nullable |

TaskReminder — `task_reminders` | Column | Type | Notes | |——|——|——-| | id | INTEGER PK AUTOINCREMENT | | | taskId | INTEGER | | | userId | TEXT | | | remindAt | INTEGER | When to show reminder | | channel | TEXT | “notification” | | isSent | INTEGER | Boolean | | createdAt | INTEGER | |

TaskLog — `task_logs` (audit trail) | Column | Type | Notes | |——|——|——-| | id | INTEGER PK AUTOINCREMENT | | | taskId | INTEGER | | | userId | TEXT | | | action | TEXT | “created”, “updated”, “completed” | | notes | TEXT | | | timestamp | INTEGER | |

DAOs (9 interfaces)

SoilDao key queries:

```
@Insert void insertAnalysis(SoilAnalysis)
@Delete void deleteAnalysis(SoilAnalysis)
@Query("SELECT * FROM soil_history WHERE user_id = :uid ORDER BY timestamp DESC")
    List<SoilAnalysis> getAnalysisForUser(String uid)
@Query("SELECT * FROM soil_history WHERE user_id = :uid ORDER BY timestamp DESC LIMIT 1")
    SoilAnalysis getLatestReportForUser(String uid)
@Query("SELECT COUNT(*) FROM soil_history WHERE user_id = :uid")
    int getCountForUser(String uid)
```

DocumentDao key queries:

```
@Query("SELECT * FROM documents WHERE user_id = :uid AND expiry_date IS NOT NULL
        AND expiry_date BETWEEN :now AND :threshold")
    List<Document> getExpiringDocumentsForUser(String uid, long now, long threshold)
```

FarmTaskDao key queries:

```
@Query("SELECT * FROM farm_tasks WHERE user_id = :uid AND due_at BETWEEN :now AND :until
      AND status != 'completed'")
List<FarmTask> getUpcomingForUser(String uid, long now, long until)
@Query("UPDATE farm_tasks SET status = 'completed', completed_at = :completedAt WHERE id = :taskId")
void markCompleted(long taskId, long completedAt)
```

9. NETWORK LAYER — ALL 6 API SERVICES

RetrofitClient.java

Singleton factory. Creates 5 Retrofit instances + shared OkHttpClient.

```
private static final OkHttpClient client = new OkHttpClient.Builder()
    .connectTimeout(30, TimeUnit.SECONDS)
    .readTimeout(30, TimeUnit.SECONDS)
    .writeTimeout(30, TimeUnit.SECONDS)
    .build();
```

All instances use same OkHttpClient -> same connection pool (efficient).

5 Retrofit instances (eagerly initialized, thread-safe, no synchronization needed):

1. soilRetrofit -> base: <https://rest.isric.org/soilgrids/v2.0/>
2. meteoRetrofit -> base: <https://api.open-meteo.com/>
3. geocodingRetrofit -> base: <https://geocoding-api.open-meteo.com/>
4. mandiRetrofit -> base: <https://api.data.gov.in/>
5. groqRetrofit -> base: <https://api.groq.com/openai/v1/>

Static factory methods: `getSoilService()`, `getWeatherService()`, `getGeocodingService()`, `getMandiService()`, `getGroqService()`

SoilApiService.java

Base URL: <https://rest.isric.org/soilgrids/v2.0/> **Auth:** None (public API) **Purpose:** Satellite-based soil property data from ISRIC World Soil Information

```
@GET("properties/query")
Call<JsonObject> getSoilProperties(
    @Query("lat") double lat,
    @Query("lon") double lon,
    @Query("property") List<String> properties, // ["nitrogen","phh2o","soc","clay"]
    @Query("depth") String depth,             // "0-5cm"
    @Query("value") String value               // "mean"
);
```

Response structure:

```
{
  "properties": {
```

```

    "layers": [
      {
        "name": "nitrogen",
        "depths": [
          {
            "label": "0-5cm",
            "values": { "mean": 85.0, "Q0.05": 60.0, "Q0.95": 110.0 }
          }
        ]
      }
    ]
  }
}

```

Unit conversions applied in ScanActivity: - nitrogen: cg/kg (centgrams per kg) -> multiply 2.0 -> approximate kg/ha - phh2o: pH × 10 -> divide 10 -> real pH - soc (soil organic carbon %): heuristic: P = 10 + (soc * 0.2) - clay (%): heuristic: K = 50 + (clay * 3.5)

OpenMeteoService.java

Base URL: <https://api.open-meteo.com/> **Auth:** None (free, no API key) **Purpose:** Agricultural weather data

```

@GET("v1/forecast?current=temperature_2m,relative_humidity_2m,rain,
      precipitation,wind_speed_10m,weather_code,soil_moisture_0_to_1cm")
Call<JsonObject> getAgroWeather(
    @Query("latitude") double lat,
    @Query("longitude") double lon
);

@GET("v1/forecast?current=temperature_2m,...&daily=temperature_2m_max,...&
      timezone=auto&forecast_days=7")
Call<JsonObject> get7DayForecast(
    @Query("latitude") double lat,
    @Query("longitude") double lon
);

```

Key fields used: - current.temperature_2m — °C - current.relative_humidity_2m — % - current.precipitation — mm - current.wind_speed_10m — km/h - current.weather_code — WMO weather interpretation code - current.soil_moisture_0_to_1cm — m³/m³

Weather codes (WMO): 0=Clear, 1-3=Partly cloudy, 45,48=Fog, 51-55=Drizzle, 61-67=Rain, 71-77=Snow, 80-82=Rain showers, 95=Thunderstorm

GeocodingService.java

Base URL: <https://geocoding-api.open-meteo.com/> **Auth:** None **Purpose:** City name -> coordinates (used by WeatherAlertsActivity for location search)

```

@GET("v1/search")
Call<JsonObject> geocode(

```

```

    @Query("name") String locationName,
    @Query("count") int count
);

```

Note: ScanActivity uses Android's built-in **Geocoder** class (not this service) for reverse geocoding (GPS -> address).

MandiApiService.java

Base URL: <https://api.data.gov.in/> **Auth:** API key from BuildConfig.DATA_GOV_API_KEY **Resource ID:** 9ef84268-d588-465a-a308-a864a43d0070 (Agmarknet commodity prices dataset)

```

@GET("resource/9ef84268-d588-465a-a308-a864a43d0070")
Call<JsonObject> getCommodityPrices(
    @Query("api-key") String apiKey,
    @Query("format") String format,           // "json"
    @Query("filters[state]") String state,
    @Query("filters[commodity]") String commodity,
    @Query("limit") int limit
);

```

Response fields used: commodity, market, min_price, max_price, modal_price, arrival_date

GroqApiService.java

Base URL: <https://api.groq.com/openai/v1/> **Auth:** Bearer token in Authorization header **Model:** llama-3.3-70b-versatile **Note:** Only used in debug builds. Release builds route through Firebase Cloud Functions backend.

```

@POST("chat/completions")
Call<JsonObject> chatCompletion(
    @Header("Authorization") String token, // "Bearer " + GROQ_API_KEY
    @Body JsonObject body
);

```

Request body structure:

```

{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {"role": "system", "content": "You are a farming expert..."},
    {"role": "user", "content": "My soil has N=120, P=25, K=300, pH=6.5..."}
  ],
  "temperature": 0.7,
  "max_tokens": 1024
}

```

ElevationApiService.java

Base URL: <https://api.open-meteo.com/> **Auth:** None

```

@GET("v1/elevation")

```



```
Call<JsonObject> getElevation(
    @Query("latitude") double lat,
    @Query("longitude") double lon
);
```

Used for terrain-aware agricultural context.

10. FIREBASE SERVICES

Firestore

Methods supported: 1. Google Sign-In (OAuth 2.0 via Play Services) 2. Phone OTP (SMS) 3. Email + Password

How Google Sign-In works technically: 1. `GoogleSignInClient` fetches ID token from Google 2. ID token -> `GoogleAuthProvider.getCredential(idToken, null)` 3. `auth.signInWithCredential(credential)` -> Firebase validates token with Google servers 4. Firebase issues its own UID independent of Google account

How Phone OTP works: 1. `PhoneAuthProvider.verifyPhoneNumber()` sends SMS via Firebase 2. User enters code -> `PhoneAuthProvider.getCredential(verificationId, code)` 3. `auth.signInWithCredential(credential)`

Session persistence: Firebase Auth automatically persists the signed-in user. `FirebaseAuth.getInstance().getCurrentUser()` returns the user across app restarts.

Firestore

Collections:

`farmers/{userId}` — User profiles - Owner-only read/write - Fields locked on create: `firstName`, `phone`, `email`, `aadhaarId`, `kisanId`, `dob`, `createdAt`, `landSize`, `cropName`, `district`, `state`, `village`, `profileImageUrl` - `createdAt` cannot be changed after create

`soilAnalysis/{docId}` — Cloud backup of soil scans - `userId` field must match auth UID - Primary storage is Room; Firestore is cloud backup

`plantAnalysis/{docId}` — Plant disease results

`crops/{cropId}` — Public crop information (read-only)

`schemes/{schemeId}` — Government scheme data (read-only)

`mandiPrices/{priceId}` — Market prices updated by Cloud Functions (read-only)

Offline persistence: Enabled in `MittiMitraApp`. Firestore caches documents locally; reads work offline. Writes are queued and synced when online.

Firestore

Usage: Profile photos only **Path:** `profile_images/{uid}/profile.jpg` **Access:** User owns their path (Storage Security Rules — not shown but follow same UID pattern)

Firebase Crashlytics

Integration: Automatic crash reporting via `firebase.crashlytics` plugin. **Custom logging:** `CrashlyticsHelper.java` wraps `FirebaseCrashlytics.getInstance().log()` and `.recordException()`. **Configuration:** `@Keep` annotation on Activity classes prevents ProGuard from obfuscating crash class names.

Firebase Analytics

Integration: `AnalyticsHelper.java` wraps `FirebaseAnalytics.getInstance(context)`. **Events logged:** scan completion, recommendation view, feature card taps, auth events.

Firebase App Check

Purpose: Prevents unauthorized API access. Only authentic, unmodified MittiMitra app instances can call Firebase services.

Debug: `DebugAppCheckProviderFactory` generates a debug token printed in Logcat. Register this in Firebase Console for dev/test.

Release: `PlayIntegrityAppCheckProviderFactory` uses Google Play Integrity API. - Checks: device integrity (rooted?), app signature (modified APK?), Play Store licensing. - Returns attestation token -> Firebase verifies before serving data.

Conditional: `BuildConfig.ENABLE_APP_CHECK` flag. False in debug (no Play Integrity available in emulator/dev), true in release.

Firebase Cloud Functions

Region: `asia-south1` (Mumbai — low latency for Indian users) **Client:** `BackendFunctionsClient.java`

Functions called:

`getSoilAdvisory(SoilAdvisoryRequest)` -> Calls Groq Llama with soil data, returns fertilizer recommendations

`getPlantDiagnosis(PlantDiagnosisRequest)` -> Calls HuggingFace vision model with plant image, returns disease diagnosis

`getCropSchedule(CropScheduleRequest)` -> Calls Groq Llama with crop + location data, returns planting schedule

`getFarmerChatResponse(ChatRequest)` -> Calls Groq Llama in chat mode, returns response

Why Cloud Functions for AI? Security: API keys (Groq, HuggingFace) never leave the server. If they were in the APK, they'd be extractable by reverse engineering. App Check: Cloud Functions can verify the App Check token, ensuring only genuine app instances can trigger AI calls.

ApiEnvelope.java: Wrapper for function responses. Has `success` boolean, `data` object, `error` string.

BackendCallback.java: Typed callback interface. `onSuccess(T data)` / `onError(Exception e)`.

StringOrListTypeAdapter: Custom Gson deserializer. Some AI responses return a string, some return a JSON array. This adapter handles both transparently.

11. ML / AI PIPELINE

TensorFlow Lite — On-Device Soil Classification

Model file: assets/soil_classifier.tflite **Input:** Float32 tensor [1, 224, 224, 3] — batch=1, height=224, width=224, RGB channels **Output:** Float32 tensor [1, 10] — 10 probability values (one per soil class) **Classes:** Alluvial, Black, Clay, Red, Sandy, Loam, Laterite, Yellow, Peaty, Chalky (must match this exact order)

Preprocessing pipeline: 1. Bitmap -> resize to 224×224 2. `getPixels()` -> int[] of ARGB values 3. Extract R, G, B -> divide by 255.0f -> 0.0–1.0 range 4. Fill ByteBuffer in RGB order (not ARGB) 5. ByteBuffer must be DIRECT (off-heap) and NATIVE_ENDIAN

Inference:

```
Interpreter interpreter = new Interpreter(loadModelFile())
float[][] output = new float[1][10]
interpreter.run(inputBuffer, output)
int maxIndex = argmax(output[0]) // getMaxIndex()
```

Model loading: MappedByteBuffer via FileChannel.map() — memory-mapped file access, zero-copy from disk. Model stays paged in RAM only while Interpreter is open.

Thread safety: Interpreter is NOT thread-safe. Each inference creates a new Interpreter instance (in try-with-resources). This is intentional.

Intelligence layer (post-inference): `applySmartCorrections()` adjusts satellite NPKP values using detected soil type + image brightness. This is the project's key innovation — combining satellite data with local visual evidence.

Groq Llama 3.3 70B — Natural Language Recommendations

Used for: - Soil advisory (fertilizer recommendations) - Chat (Kisan Sahayak) - Crop calendar generation

Why Groq: Extremely fast inference (hundreds of tokens/second). Open-source Llama model. Cost-effective.

Why 70B: Large enough for nuanced agricultural advice in multiple languages.

System prompt for soil advisory (conceptual): “You are an expert Indian agricultural scientist. Given soil nutrient data (N, P, K, pH), soil type, weather, and location, provide specific fertilizer recommendations in [user’s language]. Format: diagnosis, recommendations, expected improvement.”

HuggingFace — Plant Disease Detection

Used in: PlantScanActivity via backend **Model type:** Vision transformer or CNN fine-tuned on plant disease dataset **Input:** Base64-encoded plant image **Output:** Disease name, confidence, treatment recommendations

12. BACKGROUND PROCESSING — NotificationWorker.java

Class: NotificationWorker extends Worker **Scheduled:** Every 15 minutes via PeriodicWorkRequest in MittiMitraApp

Minimum interval: WorkManager enforces minimum 15 minutes for periodic work (OS battery optimization).

Unique work name: “DailyAlerts” with KEEP policy — prevents duplicate workers.

Responsibilities in order:

1. Weather Alerts

```
GET lat/lon from user_location_cache SharedPreferences
-> Open-Meteo API call (getAgroWeather)
-> Parse temperature, humidity
-> If humidity > 80%: show "High Humidity - Fungal Disease Risk" notification
-> If temperature > 38°C: show "Extreme Heat - Check Irrigation" notification
-> Parse sunrise/sunset from daily data -> save to AppPreferences (for auto-dark-mode)
```

2. Mandi Price Update

```
Read mandi_price_cache SharedPreferences
-> If cached prices exist: show top 3 commodities in notification
```

3. Document Expiry Checks

```
Room: getExpiringDocumentsForUser(uid, now, now + 7days)
-> For each expiring document: show notification with document name and days remaining
-> Notification ID: document ID % 1000 (modulo prevents int overflow)
```

4. Task Reminders

```
Room: getUpcomingReminders(uid, now, now + 24hours)
-> For each reminder where isSent == false:
    -> Check task status (not already completed)
    -> Show notification: task title, due time
    -> Mark reminder isSent = true in Room
```

Notification Creation

```
NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle(title)
    .setContentText(body)
    .setAutoCancel(true)
    .setContentIntent(pendingIntent) // Opens HomeActivity
    .setColor(darkMode ? Color.GREEN : Color.parseColor("#2E7D32"))
```

Channel: “mitti_mitra_alerts” (IMPORTANCE_DEFAULT)

Android 13+ guard: Checks `POST_NOTIFICATIONS` permission before building notification. `ContextCompat.checkSelfPermission()`.

Dark mode awareness: `AppPreferences.isDarkMode()` -> selects appropriate notification accent color.

13. REPOSITORY PATTERN — DATA LAYER

Overview

Three repositories implement the Repository Pattern: - View/Activity -> Repository interface (domain layer) - Repository -> Firebase/Room (data layer) This decouples UI from data source. Swap Firestore for any other backend without changing Activities.

Domain Interfaces

PredictionRepository.java (interface in domain/repository/)

```
void fetchSoilAdvisory(SoilAdvisoryRequest req, BackendCallback<SoilAdvisoryData> callback)
void fetchPlantDiagnosis(PlantDiagnosisRequest req, BackendCallback<PlantDiagnosisData> callback)
void fetchCropSchedule(CropScheduleRequest req, BackendCallback<CropScheduleData> callback)
void fetchChatResponse(ChatRequest req, BackendCallback<ChatResponseData> callback)
```

TaskRepository.java (interface in domain/repository/)

```
long createField(Field field)
long createTask(FarmTask task)
void updateTask(FarmTask task)
void completeTask(long taskId, long completedAt)
void upsertReminder(TaskReminder reminder)
List<FarmTask> getTasksForUser(String userId)
List<FarmTask> getUpcomingTasks(String userId, long now, long until)
List<TaskReminder> getUpcomingReminders(String userId, long now, long until)
List<FarmTask> getTaskTemplatesByCropStage(String crop, String stage, String userId)
```

UserProfileRepository.java (interface in domain/repository/)

```
void saveProfile(Map<String, Object> data, callback)
void loadProfile(String userId, callback)
void updateProfilePhoto(Uri imageUri, callback)
```

Implementations

FirebasePredictionRepository.java

Primary path: `BackendFunctionsClient` -> Firebase Cloud Functions Fallback path (debug only): `GroqApiService` -> direct Groq API

Fallback trigger conditions (Firebase error codes): - `NOT_FOUND` — Function not deployed - `UNAVAILABLE` — Firebase Functions unreachable - `DEADLINE_EXCEEDED` — Timeout - `INTERNAL` — Function crashed - `UNKNOWN_ERROR` — Catch-all

RoomTaskRepository.java

Wraps all Room DAO calls. Adds: - Auto-logging: `createTask()` and `updateTask()` automatically create `TaskLog` entries - `completeTask()` logs completion with timestamp

FirebaseUserProfileRepository.java

Wraps Firestore profile operations. Handles: - Phone uniqueness check (with `PERMISSION_DENIED` fallback — Cloud Function responsibility) - Profile photo upload to Firebase Storage + URL save to Firestore

14. SECURITY ARCHITECTURE

Layers of Security

Layer 1 — Firebase App Check Verifies the calling app is authentic MittiMitra, not a cloned or modified APK. Release: Play Integrity API (Google's device attestation) Debug: Debug token (whitelisted in Firebase Console)

Layer 2 — Firebase Auth UID Every Firestore operation is authorized against the authenticated UID. No UID = no access.

Layer 3 — Firestore Security Rules Server-side enforcement. Even if App Check passes, data is still UID-locked:

```
match /farmers/{userId} {
  allow read: if request.auth.uid == userId;
  allow create: if request.auth.uid == userId && valid fields;
  allow update: if request.auth.uid == userId && !change createdAt;
  allow delete: if request.auth.uid == userId;
}
```

Layer 4 — AI API Keys in Backend Only Groq and HuggingFace API keys are in Firebase Cloud Functions environment variables. They never appear in the APK. In debug, Groq key is in BuildConfig (from `local.properties`, never committed to git).

Layer 5 — Encrypted SharedPreferences `androidx.security:security-crypto` — `EncryptedSharedPreferences` for sensitive tokens. Uses AES-256 for values, AES-256-SIV for keys.

Layer 6 — FileProvider Prevents raw `file://` URI exposure. Camera images shared via `content://` URI with temporary permissions.

Layer 7 — ProGuard Obfuscation Release APK has obfuscated class/method names. Makes reverse engineering harder.

Layer 8 — API Keys in local.properties (never in git) `local.properties` is in `.gitignore`. Keys never committed. CI/CD injects keys as environment variables during build.

Authentication Flow Security

Phone OTP flow: - Firebase validates the SMS OTP server-side. The app never sees the raw SMS.
- `verificationId` is a server-issued token, useless without the correct OTP.

Google Sign-In: - ID token is a short-lived JWT signed by Google. - Firebase validates signature against Google's public keys. - The actual Google account password never touches the app.

15. UTILITY CLASSES

SessionManager.java

SharedPreferences wrapper for user session. Keys: `USER_ID`, `USER_NAME`, `IS_GUEST` Methods: `saveUser()`, `clearSession()`, `isLoggedIn()`, `getUserId()`, `getUserName()`, `isGuest()` Stored in `MittiMitraSession` preferences file.

AppPreferences.java

SharedPreferences wrapper for app settings. Keys: `FONT_SCALE`, `APP_LANGUAGE`, `DYSLEXIC_FONT`, `APP_THEME`, `LAST_LAT`, `LAST_LON`, `LAST_LOC_NAME`, `LAST_CROP`, `LAST_SOIL_TYPE`, `LAST_FIELD_SIZE`, `AUTO_DARK_MODE`, `LAST_SUNRISE`, `LAST_SUNSET`, `ONBOARDING_DONE`

Nuance: `LAST_LAT`/`LAST_LON` stored as String (not float) to preserve full double precision. Float has ~7 significant digits; GPS coordinates need ~10. Legacy Float values are migrated: `getLastLocation()` catches `ClassCastException` and re-saves as String.

UserIdentityResolver.java

Resolves the “who is the current user” question across auth states.

`getActiveUserId(Context)`: - If Firebase user logged in: returns Firebase UID - Else: returns session-saved `userId` - Never returns null

`getActiveUserIdOrCreateGuest(Context)`: - Same as above, but if no session: creates local guest identity

`createOrRestoreLocalGuestIdentity(Context)`: - Guest ID format: “guest_local_{sha256_hash}”
- Hash input: `ANDROID_ID` + “|” + `LOCAL_IP` - Display name: “Guest {IP}” or “Guest Farmer” - Stable across sessions (same device = same guest ID)

NutrientStatus.java

Maps nutrient values to status labels and colors.

```
getNitrogenStatus(double n):  
    < 60 kg/ha: LOW (red)  
    60-120: OK (green)  
    > 120: HIGH (orange)
```

```
getPhosphorusStatus(double p):  
    < 10 kg/ha: LOW
```

```
10-25: OK
> 25: HIGH
```

```
getPotassiumStatus(double k):
    < 100 kg/ha: LOW
    100-280: OK
    > 280: HIGH
```

```
getPhStatus(double pH):
    < 6.0: ACIDIC (red)
    6.0-7.5: NEUTRAL (green)
    > 7.5: ALKALINE (orange)
```

Returns `Status` enum with `labelResId` (R.string) and `color` (int ARGB).

SoilNutrientMapper.java

Maps soil type string to typical P and K ranges when SoilGrids data unavailable. Fallback heuristic based on Indian soil science literature.

BitmapUtils.java

`saveBitmapToInternalStorage(Bitmap, Context, String filename):` - `FileOutputStream` in `try-with-resources` - JPEG compression at quality 90

ImageCompressor.java

`compress(Bitmap, int targetWidth, int quality):` - Scales bitmap maintaining aspect ratio - JPEG encode to `ByteArrayOutputStream` - Returns compressed `byte[]` - Two `InputStream` closures converted to `try-with-resources`

NetworkUtils.java

`isNetworkAvailable(Context):` - `ConnectivityManager.getActiveNetworkInfo()` - Returns `boolean`

AnalyticsHelper.java

Wraps Firebase Analytics. `init(Application)` -> stores `FirebaseAnalytics` instance. `logEvent(String name, Bundle params)` -> fires analytics event. `logScreen(Activity)` -> logs screen view.

CrashlyticsHelper.java

Wraps `FirebaseCrashlytics`. `log(String message)` -> non-fatal log. `recordException(Throwable t)` -> records exception with context. `setUserId(String uid)` -> associates crashes with user.

SchemeRepository.java

Loads government scheme data from `assets/schemes_data.json`. Fallback: Firestore `schemes` collection. `loadFromAssets()` — `InputStream` in `try-with-resources`. All 4 catch blocks use `Log.e()` (not `e.printStackTrace()`).

SoilDataManager.java

Manages SoilGrids data caching and retrieval. `BufferedReader` in try-with-resources for CSV parsing. `Log.e()` in all catch blocks.

ErrorHandler.java

Centralized error classification and user-friendly message mapping. Maps exception types to string resource IDs.

Result.java

Generic sealed-class-style wrapper: `Result<T>` with `Success<T>` and `Error` subtypes. Used for repository return types.

WeatherUtils.java (referenced but path not shown)

`getWeatherDescription(int code) -> String[2]: {emoji, description}` `getAgriculturalRecommendations(double temp, int humidity, double wind, double precip) -> List alerts`

LocationContextResolver.java

Resolves location context for AI prompts. Combines GPS location + district name into structured context object.

16. LOCALIZATION — 14 LANGUAGES

Implementation

Android resource qualifier system: `values-{language_code}/strings.xml`

<code>values/</code>	English (default)
<code>values-hi/</code>	Hindi
<code>values-ta/</code>	Tamil
<code>values-te/</code>	Telugu
<code>values-kn/</code>	Kannada
<code>values-mr/</code>	Marathi
<code>values-bn/</code>	Bengali
<code>values-gu/</code>	Gujarati
<code>values-pa/</code>	Punjabi
<code>values-as/</code>	Assamese
<code>values-mni/</code>	Manipuri (Meitei script)
<code>values-brx/</code>	Bodo
<code>values-lus/</code>	Mizo
<code>values-grt/</code>	Garó
<code>values-night/</code>	Dark mode overrides

How switching works: 1. User picks language in `LanguageActivity` 2. Language code saved to `AppPreferences` 3. `BaseActivity.attachBaseContext()` is called before every `Activity` creates 4. In-

side: Locale set, Configuration updated, Context wrapped with ContextThemeWrapper 5. Activity creates with new locale -> all getString() calls resolve correct language file

RTL support: android:supportsRtl="true" in Manifest. Arabic/Urdu not currently in list but architecture supports it.

String references for dynamic content: All user-visible strings use R.string.* — no hardcoded English in Java code. This is enforced throughout the codebase (major cleanup done across all files).

17. DATA FLOW — END TO END (SCAN TO RECOMMENDATION)

User taps "Scan Soil" card on HomeActivity

v

ScanActivity.onCreate()

- > fusedLocationClient.getLastLocation()
- > fetchAddress() [background: geocoder]
- > fetchAgroData() [Retrofit -> Open-Meteo]
- > fetchLiveSoilData() [Retrofit -> SoilGrids]

v

User selects/captures soil image

- > cameraLauncher or galleryLauncher
- > decodeSampledBitmap(uri, 1024) [memory-safe decode]
- > currentImageBitmap set
- > "Analyze" button appears

v

User taps "Analyze"

- > startAnalysis()
- > analysisExecutor.execute(runLocalInference)

v

runLocalInference() [background thread]

- > loadModelFile() [mmap from assets]
- > Bitmap resized to 224x224
- > Pixel normalization to ByteBuffer
- > interpreter.run() [TFLite inference]
- > getMaxIndex() [argmax]
- > detectedSoilType = SOIL_LABELS[maxIndex]

v

generateSmartReport() [background thread]

- > applySmartCorrections(detectedSoilType)
 - > soil type heuristics (N/P/K factors)
 - > calculateBrightness(bitmap) -> adjust N
- > Build JSONObject: {N, P, K, pH, weather, soil_dynamic, location, detected_soil, user_not
- > SoilAnalysis entity created
- > UserIdentityResolver.getActiveUserId() -> userId
- > soilDao().insertAnalysis(analysis) [Room write]
- > AppPreferences.setLastSoilType()

```

v
Main thread: startActivity(RecommendationActivity)
-> intent extra: DETECTED_SOIL_TYPE
v
RecommendationActivity.onCreate()
-> loadLatestReport() [Room read via dbExecutor]
-> Parse JSON: extract N, P, K, pH
-> calculateSoilHealthScore(pH)
-> Display nutrient cards with NutrientStatus colors
-> Display health score gauge
v
fetchAdvisory() [Room JSON -> backend call]
-> FirebasePredictionRepository.fetchSoilAdvisory()
-> BackendFunctionsClient.getSoilAdvisory()
-> Firebase Functions (asia-south1)
-> Cloud Function: Groq Llama 3.3 70B API call
-> Returns: SoilAdvisoryData {advisory text, crop recommendations}
-> Display advisory text in card
v
suggestTasks() [dbExecutor]
-> TaskSuggestionEngine.suggestFromSoilAdvisory()
-> Creates FarmTask entries in Room
-> Creates TaskReminder entries (6-12hrs before due)
v
User can:
-> Download PDF (PdfDocument API)
-> Share PDF (FileProvider + ACTION_SEND)
-> Hear TTS narration (TextToSpeech)
-> Navigate back to History

```

18. IMPORTANT APIs USED AND WHY

ISRIC SoilGrids v2.0

URL: <https://rest.isric.org/soilgrids/v2.0/> **Why:** Only free global soil database with REST API. Provides satellite-derived soil properties (nitrogen, pH, organic carbon, clay content) for any GPS coordinate on Earth. No API key required. Data resolution: 250m grid. **Limitation:** These are statistical estimates (mean of satellite/model ensemble), not lab measurements. The app supplements with visual TFLite classification and applies correction factors.

Open-Meteo

URL: <https://api.open-meteo.com/> **Why:** Completely free, no API key, agriculture-specific variables (soil moisture at multiple depths, evapotranspiration, UV index). Based on ERA5 and high-res NWP models. Provides soil_moisture_0_to_1cm which is unique to agricultural forecasting APIs.

data.gov.in / Agmarknet

URL: <https://api.data.gov.in/> **Resource:** 9ef84268-d588-465a-a308-a864a43d0070 **Why:** Official Indian government open data. Agmarknet collects daily commodity arrival and price data from 3,000+ APMC markets across India. Free API with registration. Used for mandi prices — gives farmers real market data to sell at best price.

Groq Cloud (Llama 3.3 70B)

Why: Fastest available inference for a 70B parameter model. Provides intelligent, contextual agricultural advice that understands Indian farming context, multiple crops, soil types, and regional variations. 70B size provides better agronomy knowledge than smaller models.

HuggingFace (via Backend)

Why: Pre-trained plant disease detection models. Eliminates need to train our own model. HuggingFace Inference API provides easy access to state-of-the-art vision models.

TensorFlow Lite (Local ML)

Why: On-device inference. No network needed for soil classification. Works in areas with poor connectivity (common in rural India). Zero latency for classification (ms vs seconds). Model file: soil_classifier.tflite (224×224 input, 10-class output).

Firebase Cloud Functions

Why: Secure server-side AI key management. Functions run in Google's infrastructure, protected by App Check. Keys never leave the server. Also allows backend logic changes without app updates.

Firebase App Check with Play Integrity

Why: Prevents API abuse. Without App Check, anyone with the google-services.json could call Firebase services. Play Integrity ties API access to genuine, unmodified app instances on real Android devices.

19. DEPLOYMENT — BUILD, SIGN, RELEASE

Local Debug Build

`./gradlew assembleDebug`

- Uses debug keystore (auto-generated by Android Studio at ~/.android/debug.keystore)
- No ProGuard
- App Check disabled
- Groq API key injected from local.properties
- Output: app/build/outputs/apk/debug/app-debug.apk

Local Release Build

Prerequisites: 1. `keystore.properties` file (gitignored) with: `storeFile=path/to/mittimitra.jks`
`storePassword=...` `keyAlias=...` `keyPassword=...` 2. `local.properties` with:
`OPENWEATHER_API_KEY=...` `DATA_GOV_API_KEY=...` `BACKEND_BASE_URL=https://asia-south1-mitti-m`

`./gradlew assembleRelease`

- Signs with production keystore
- ProGuard: minifies + obfuscates + shrinks resources
- App Check enabled
- Groq key empty (backend only)
- Output: `app/build/outputs/apk/release/app-release.apk`

Release AAB (for Play Store)

Google Play requires AAB format (not APK) for new apps.

`./gradlew bundleRelease`

Output: `app/build/outputs/bundle/release/app-release.aab`

Play Store uses App Signing by Google: you upload AAB, Google signs with its own key before delivering to devices.

Keystore

`mittimitra.jks` — Java KeyStore format - Created once: `keytool -genkey -v -keystore mittimitra.jks -keyalg RSA -keysize 2048 -validity 10000 -alias mittimitra - SHA-1 fingerprint registered in: - Firebase Console (for Google Sign-In OAuth) - Google Cloud Console (for Play Services) - google-services.json (multiple certificate entries for debug + release)`

Critical: If keystore is lost, you CANNOT update the app on Play Store. Back it up externally.

Firebase Console Setup

1. Create project: `mitti-mitra-4ee33`
2. Add Android app with package name `in.mittimitra.app`
3. Download `google-services.json` -> place in `app/` folder
4. Register SHA-1 fingerprints (both debug and release)
5. Enable Auth providers: Google, Phone, Email/Password
6. Enable Firestore -> deploy security rules from `firestore.rules`
7. Enable Storage -> set storage rules
8. Enable App Check -> register Play Integrity for release
9. Enable Cloud Functions -> deploy functions from `functions/` directory
10. Enable Crashlytics, Analytics (auto via plugin)

GitHub Actions CI/CD

File: `.github/workflows/android-ci.yml`

Runs on push/PR to main: - Checkout code - Set up JDK 17 - Inject secrets (API keys) as environment variables - Run `./gradlew test` (unit tests) - Run `./gradlew lintRelease` (lint check against baseline) - Run `./gradlew assembleRelease` (build APK)

Secrets stored in GitHub Repository Settings -> Secrets and Variables -> Actions.

Versioning

Current: versionCode=5, versionName="1.0.5" History: - 1.0.0 (code 1): Initial release - 1.0.1 (code 2): Bug fixes - 1.0.2 (code 3): Version bump - 1.0.3 (code 4): More fixes - 1.0.5 (code 5): Compliance + production changes

Note: versionCode 4 not seen in git log (may have been skipped or internal).

Rule: versionCode must strictly increase. versionName is for humans, versionCode is for Play Store.

20. FUNCTION-TO-FUNCTION MAPPING

User Scan Flow (complete call chain)

```
HomeActivity.setupGridNavigation() -> click card_scan -> startActivity(ScanActivity)
ScanActivity.onCreate() -> checkLocationAndFetchData()
checkLocationAndFetchData() -> fusedLocationClient.getLastLocation()
    -> onSuccess: fetchAddress(lat, lon)
        fetchAgroData(lat, lon)
        fetchLiveSoilData(lat, lon)

fetchAddress() -> analysisExecutor.execute() -> Geocoder.getFromLocation()
    -> Handler(Looper.getMainLooper()).post() -> chipLocation.setText()

fetchAgroData() -> RetrofitClient.getWeatherService().getAgroWeather().enqueue()
    -> onResponse() -> WeatherUtils.getWeatherDescription()
        -> WeatherUtils.getAgriculturalRecommendations()
        -> updateDashboardUI()

fetchLiveSoilData() -> RetrofitClient.getSoilService().getSoilProperties().enqueue()
    -> onResponse() -> getMeanValue(layer) [per layer]
        -> finalN, finalP, finalK, finalpH set
        -> runOnUiThread() -> updateDashboardUI()

[user captures image]
cameraLauncher.result -> decodeSampledBitmap(uri, 1024)
    -> BitmapFactory.decodeStream() x2 [bounds + actual]
    -> currentImageBitmap.set -> onImageSuccess()

btnAnalyze.onClick -> startAnalysis()
startAnalysis() -> analysisExecutor.execute(runLocalInference)
```

```

runLocalInference() -> loadModelFile() -> AssetFileDescriptor -> FileChannel.map()
    -> Bitmap.createScaledBitmap(224, 224)
    -> ByteBuffer fill (pixel normalization)
    -> Interpreter.run()
    -> getMaxIndex(output) -> SOIL_LABELS[maxIndex]
    -> generateSmartReport(detectedSoil, userNotes)

generateSmartReport() -> analysisExecutor.execute():
    -> applySmartCorrections(soilType)
        -> calculateBrightness(bitmap) [50x50 center sample]
    -> JSONObject build
    -> UserIdentityResolver.getActiveUserId()
    -> soilDao().insertAnalysis(SoilAnalysis)
    -> AppPreferences.setLastSoilType()
    -> runOnUiThread() -> startActivity(RecommendationActivity)

RecommendationActivity.onCreate() -> dbExecutor.execute(loadLatestReport)
loadLatestReport() -> soilDao().getLatestReportForUser(userId)
    -> parse JSON [N, P, K, pH, detected_soil]
    -> runOnUiThread() -> populate nutrient cards
        -> calculateSoilHealthScore(pH) -> health gauge

fetchAdvisory() -> FirebasePredictionRepository.fetchSoilAdvisory(request, callback)
    -> BackendFunctionsClient -> FirebaseFunctions.getHttpsCallable("getSoilAdvisory")
    -> .call(requestData)
    -> onSuccess() -> ApiEnvelope parse -> SoilAdvisoryData
    -> (fallback if error) -> GroqApiService.chatCompletion() [debug only]
    -> callback.onSuccess(data)
    -> runOnUiThread() -> display advisory text

suggestTasks() -> dbExecutor.execute():
    -> TaskSuggestionEngine.suggestFromSoilAdvisory(advisory, userId)
    -> RoomTaskRepository.createTask(farmTask) × N
    -> RoomTaskRepository.upsertReminder(reminder) × N

```

Notification Flow

```

MittiMitraApp.onCreate() -> WorkManager.enqueueUniquePeriodicWork("DailyAlerts")
    [every 15 minutes]
    -> NotificationWorker.doWork()
    -> getSharedPreferences("user_location_cache") -> lat, lon
    -> RetrofitClient.getWeatherService().getAgroWeather() [sync HTTP]
    -> parse -> show weather notification if alert
    -> getSharedPreferences("mandi_price_cache") -> prices -> notification
    -> MittiMitraDatabase.getDatabase() -> documentDao().getExpiringDocumentsForUser()
    -> per expiring doc -> NotificationManagerCompat.notify()
    -> farmTaskDao().getUpcomingForUser() + taskReminderDao()
    -> per reminder -> notify() -> taskReminderDao().markSent()

```

21. IMPORTANT FUNCTIONS LIST

Function	File	What it does
MittiMitraApp.onCreate()	MittiMitraApp.java	App init: Firebase, Analytics, AppCheck, Firestore offline, WorkManager
ScanActivity.checkLocationAndFetchData()	ScanActivity.java	Gets GPS, triggers parallel API calls
ScanActivity.fetchLiveSoilData()	ScanActivity.java	ISRIC SoilGrids API call, parses N/P/K/pH
ScanActivity.runLocalInference()	ScanActivity.java	TFLite model inference, soil classification
ScanActivity.applySmartCorrectScans()	ScanActivity.java	Adjusts satellite data using visual + type heuristics
ScanActivity.calculateBrightness()	ScanActivity.java	Image center luminance for organic matter estimate
ScanActivity.generateSmartReport()	ScanActivity.java	Builds final report JSON, saves to Room
ScanActivity.decodeSampledBitmap()	ScanActivity.java	Memory-safe image decode with subsampling
ScanActivity.loadModelFile()	ScanActivity.java	Memory-maps TFLite model from assets
RecommendationActivity.calculateSoilHealthScore()	RecommendationActivity.java	Weighted health score 0-100
RecommendationActivity.fetchAIRecommendation()	RecommendationActivity.java	Triggers AI recommendation via backend
FirebasePredictionRepository.fetchSoilPrediction()	FirebasePredictionRepository.java	Backend Functions call + Groq fallback
MittiMitraDatabase.getDatabase()	MittiMitraDatabase.java	Thread-safe singleton DB access
UserIdentityResolver.getActiveUserId()	UserIdentityResolver.java	Fetches Firebase UID or guest ID resolution
AppPreferences.getLastLocation()	AppPreferences.java	Float->String migration for GPS precision
NotificationWorker.doWork()	NotificationWorker.java	Weather/mandi/expiry/reminder notifications
TaskSuggestionEngine.suggestFarmTasks()	TaskSuggestionEngine.java	Auto-creates farm tasks from AI output
RoomTaskRepository.createTask()	RoomTaskRepository.java	Task insert + auto-log
LoginActivity.firebaseAuthWithGoogle()	LoginActivity.java	Google OAuth -> Firebase credential
LoginActivity.applyPendingPhoneLink()	LoginActivity.java	Resolves duplicate account phone link
ProfileActivity.checkPhoneUniqueness()	ProfileActivity.java	Duplicate phone guard before save
HomeActivity.refreshUserNameFromFirestore()	HomeActivity.java	Refreshes name from Firestore on resume
BackendFunctionsClient.getSoilData()	BackendFunctionsClient.java	Backend Functions invocation
SoilNutrientMapper.getNPKForSoilType()	SoilNutrientMapper.java	Heuristic P/K values per soil type
NutrientStatus.getNitrogenStatus()	NutrientStatus.java	N value -> color + label
SchemeRepository.loadFromAssets()	SchemeRepository.java	JSON parse from assets (try-with-resources)
ImageCompressor.compress()	ImageCompressor.java	Bitmap -> compressed JPEG bytes
BitmapUtils.saveBitmapToInternalStorage()	BitmapUtils.java	Save bitmap with try-with-resources
WeatherUtils.getAgriculturalRecommendations()	WeatherUtils.java	Weather -> actionable farm alerts

22. PROJECT NUANCES AND GOTCHAS

1. Two Package Names

- namespace "com.mittimitra" — Java package, used in code
- applicationId "in.mittimitra.app" — Play Store ID
- FileProvider authority: `${applicationId}.provider` -> `in.mittimitra.app.provider` (release)
- Firebase SHA-1 registered for `in.mittimitra.app`

2. Kotlin Plugin on Java Project

Kotlin plugin present in plugins but no .kt files in main source. AGP 8.x requires Kotlin plugin even for Java-only projects because it's part of the build pipeline. Not a mistake.

3. SoilGrids JSON Array — The Critical Fix

Early version crashed because `layers` was treated as a `JsonObject`. It's a `JsonArray`. The fix loops: `for (JsonElement layerElement : layers) { JsonObject layer = layerElement.getAsJsonObject(); }`.

4. JSON Key Mismatch Bug (Fixed)

`ScanActivity` stores JSON with short keys: "N", "P", "K", "pH". Early `CompareActivity` and `HistoryActivity` used full names: "nitrogen", "phosphorus", etc. Fix: `CompareActivity.java` and `HistoryActivity.java` updated to use short keys to match `ScanActivity` output.

5. LinearLayoutManager Before setAdapter (Two Crashes)

`SettingsActivity` and `DocumentsActivity` both had this crash:

```
// WRONG - crashes NullPointerException
recyclerView.setAdapter(adapter);
recyclerView.setLayoutManager(new LinearLayoutManager(this));

// CORRECT
recyclerView.setLayoutManager(new LinearLayoutManager(this));
recyclerView.setAdapter(adapter);
```

`RecyclerView` needs a `LayoutManager` before it can measure item positions for adapter.

6. GPS Precision Loss Bug (Fixed)

`SharedPreferences.putFloat()` for lat/lon causes precision loss (float = 7 significant digits, GPS needs ~10). Fixed: store as String. `AppPreferences.getLastLocation()` catches `ClassCastException` when reading legacy Float values and migrates them to String.

7. Room PK Field Name

`SoilAnalysis` entity: primary key field is `analysisId`, not `id`. `HistoryActivity.java` used `analysis.id` (crash) — fixed to `analysis.analysisId`.

8. Thread Safety in ScanActivity

All Room writes happen in `analysisExecutor`. All UI updates go through `runOnUiThread()` or `Handler(Looper.getMainLooper()).post()`. TFLite Interpreter is created and closed per inference in try-with-resources. This prevents ANR and race conditions.

9. Duplicate WorkManager Scheduling (Fixed)

`HomeActivity` previously called `WorkManager.enqueue()` on every launch. This was moved to `MittiMitraApp.onCreate()` with `ExistingPeriodicWorkPolicy.KEEP` — no duplicate scheduling regardless of how many times the app is opened.

10. Phone Uniqueness Enforcement

`ProfileActivity` checks Firestore for duplicate phone before saving. But Firestore security rules deny client-side collection listing. The `PERMISSION_DENIED` error is intentionally caught and falls through to `doFirestoreSave()`. The real enforcement is meant to be a Cloud Function. This is a known architectural gap documented in memory.

11. Firebase App Check + Emulator

App Check with Play Integrity fails on emulators (no Play Store). During development, `ENABLE_APP_CHECK=false` in debug builds bypasses this. For testing App Check itself, use `DebugAppCheckProviderFactory` and register the debug token in Firebase Console.

12. Firestore Field Locked at Create

Firestore rule prevents `createdAt` from being changed after initial create:

```
allow update: if !request.resource.data.diff(resource.data).affectedKeys().hasAny(['createdAt'])
```

This enforces data integrity — creation timestamp is immutable.

13. TFLite + Main Thread (ANR Fix)

Original code ran TFLite on main thread. On slow devices, 224×224 inference can take 500ms+, blocking the UI and causing ANR (Application Not Responding). Fix: `analysisExecutor.execute()` moves inference to background thread.

14. TextToSpeech Locale

TTS uses the user's selected app language from `AppPreferences`. If TTS engine doesn't support the language, it falls back to English silently.

15. Notification ID Overflow

Document expiry notification IDs were `(int) document.documentId` which could overflow int range for large IDs. Fix: `(int)(document.documentId % 1000)` — stable, non-overflowing.

16. Dark Mode Auto-Switching

`NotificationWorker` saves sunrise/sunset times from Open-Meteo weather response to `AppPreferences`. `AppPreferences.isAutoNightMode()` compares current time to these values. `BaseActivity` then applies dark/light theme automatically at sunset/sunrise without user interaction.

17. Firestore Offline Mode + `PersistentCacheSettings`

New Firestore SDK uses `PersistentCacheSettings.newBuilder().build()` via `setLocalCacheSettings()`. Old SDK used `setPersistenceEnabled(true)`. The code uses reflection to try new API first, falls back to old one. This maintains compatibility across Firebase SDK versions without separate APKs.

18. Guest User Architecture

Guest users get a deterministic ID derived from `ANDROID_ID` + IP hash. Same device = same guest ID across reinstalls (until factory reset). This lets guests retain their scan history even without auth. Data stored only locally in Room (not Firestore).

19. Yoda Conditions

`"value".equals(variable)` instead of `variable.equals("value")` prevents `NullPointerException` if variable is null. `ThemeSettingsActivity` uses this pattern.

20. try-with-resources Enforcement

All `InputStream`, `OutputStream`, `FileWriter`, `BufferedReader` instances use try-with-resources. This guarantees `close()` is called even if an exception occurs, preventing file descriptor leaks. Major cleanup done across: `BitmapUtils`, `ImageCompressor`, `SoilDataManager`, `SchemeRepository`, `DocumentsActivity`, `TipActivity`, `ScanActivity`, `HistoryActivity`.

23. VIVA QUESTIONS PREP

Q: What is the `applicationId` vs `namespace`? A: `namespace` is the Java package name used for generated R class and `BuildConfig`. `applicationId` is the unique identifier on Google Play Store. They can differ — allows rebranding code package without affecting Play identity. Our `namespace` is `com.mittimitra`, `applicationId` is `in.mittimitra.app`.

Q: How does TFLite model work? A: We load `soil_classifier.tflite` from assets as a `MappedByteBuffer`. The model was trained on Indian soil images with 10 classes. Input: 224×224 RGB image normalized to 0-1 float values packed in `ByteBuffer`. Output: array of 10 probability values. We take `argmax` (highest probability) as the classification.

Q: How do you prevent ANR? A: Any operation > 16ms should not run on main thread. We use `ExecutorService` for TFLite inference, Room database reads/writes, network response parsing, geocoding, and file I/O. Results post back to main thread via `runOnUiThread()` or `Handler`.

Q: What is Firebase App Check? A: A layer that verifies API calls come from genuine, unmodified MittiMitra app instances. Release builds use Google Play Integrity API — it checks device integrity and app signature. This prevents unauthorized access even if someone extracts the `google-services.json`.

Q: How do Firestore security rules work? A: Rules are evaluated server-side for every read/write request. They use `request.auth.uid` to verify the caller's identity. Our rules ensure users can only access their own data. The `hasOnly(...)` check on create prevents extra fields.

Q: Why are API keys in Cloud Functions? A: APKs can be decompiled. If Groq or HuggingFace keys were in the APK (even in BuildConfig), they'd be extractable. Cloud Functions run on Google's servers — keys are in Function environment variables, never exposed to client apps.

Q: What is Room migration? A: When you add a new table or column to Room, you must provide a Migration that describes the SQL changes. Without it, Room would crash on upgrade. We have migrations v2->v3 through v7->v8, each adding new tables incrementally.

Q: How does language switching work? A: User selects language -> language code saved to SharedPreferences -> `BaseActivity.attachBaseContext()` called before each Activity creates -> we create a new Locale, update Configuration, wrap Context -> all `getString()` calls use the new locale to look up the correct values-`{lang}/strings.xml` file.

Q: How does offline mode work? A: Firestore offline persistence enabled — it caches all read documents locally. Room is the primary offline database. Open-Meteo and SoilGrids data cached in SharedPreferences. TFLite runs on-device — no network needed for soil classification. The app degrades gracefully: shows cached data when offline.

Q: What is the duplicate account problem? A: A user who signed up via phone OTP later signs in with Google using the same email. Firebase creates two different Auth accounts (different UIDs). Our code detects this scenario: if the Google account email matches a phone-linked farmer profile, we delete the empty new Google account and force the user to use the original phone account. The phone number is preserved by storing it in SharedPreferences as a “pending link” applied on re-sign-in.

Q: Why use WorkManager instead of AlarmManager? A: WorkManager handles constraints (network, battery), survives device reboots, handles API level differences, and is battery-optimized. AlarmManager is lower-level and requires manual persistence across reboots.

Q: How does soil nutrient calculation work? A: Nitrogen and pH come directly from SoilGrids satellite data (with unit conversion). Phosphorus and Potassium are derived heuristically from organic carbon (SOC) and clay content respectively — SoilGrids provides SOC and clay which correlate with P and K availability. These values are then adjusted by soil type factors and image brightness analysis.

Q: What is FileProvider and why needed? A: Android 7.0+ prohibits sharing `file://` URIs between apps for security. FileProvider converts a file path to a `content://` URI and grants temporary read permission to the receiving app (camera). Without it, the camera intent cannot write the full-resolution photo to our specified location.

Q: How does the double-back-press-to-exit work? A: `System.currentTimeMillis()` is stored on first back press. On second back press, if the time difference is less than 2000ms, we call `finishAffinity()` to close all activities. Otherwise, show toast and record new timestamp.

Q: What security risk does `ENABLE_APP_CHECK = false` in debug create? A: In debug builds, Firebase services can be called by any app or tool that has the `google-services.json` and knows the collection names. This is acceptable for development but the debug keystore's SHA-1 should NOT be registered in Firebase Production project — only the release keystore SHA-1.

Q: How does the smart correction factor work mathematically? A: Each soil type has empirical N/P/K multipliers based on Indian soil science. E.g., Black soil (Regur) has `nFactor=1.25`

because it's a heavy clay soil known for high nutrient retention. Sandy soil has 0.7-0.8 because nutrients leach quickly. Additionally, image center brightness (0-255) adjusts N by $\pm 15\%$ since dark soil indicates more organic matter $\rightarrow$ higher available nitrogen. $\text{Final} = \text{satelliteValue} \times \text{soilFactor} \times \text{brightnessFactor}$.

Q: What is the `versionCode` vs `versionName` difference? A: `versionCode` is an integer that the Play Store uses to determine update order — must increase monotonically. `versionName` is a human-readable string (e.g., "1.0.5") shown to users. Play Store compares `versionCode`; users see `versionName`.

Q: Why is `exportSchema = false` in Room? A: When true, Room exports JSON schema files to disk for migration validation. Setting false disables this. It means no schema validation tooling but no extra file management. The trade-off is acceptable for a student project but not recommended for production.

Q: What does `shrinkResources true` do? A: Removes unused resources (layout XMLs, drawables, strings) from the release APK. Works in conjunction with `minifyEnabled true` (ProGuard). Together they significantly reduce APK size.

APPENDIX: KEY STRING RESOURCES

```
R.string.app_name = "Mitti Mitra"
R.string.scan_btn_analyze = "Analyze Soil"
R.string.scan_btn_analyzing = "Analyzing..."
R.string.scan_image_required = "Please select a soil image"
R.string.scan_locating = "Locating..."
R.string.alert_connecting = "Connecting..."
R.string.alert_offline_mode = "Offline Mode"
R.string.alert_data_received = "Data Received"
R.string.alert_data_unavailable = "Data Unavailable"
R.string.alert_network_error = "Network Error"
R.string.alert_too_dark = "Too dark for accurate scan. Move to better light."
R.string.status_calculating = "Calculating..."
R.string.msg_back_to_exit = "Press back again to exit"
R.string.msg_camera_permission_needed = "Camera permission required"
R.string.health_excellent = "Excellent Condition"
R.string.health_good = "Good Condition"
R.string.health_attention = "Needs Attention"
R.string.health_poor = "Poor Condition"
R.string.mandi_fallback_msg = "..." (format string with 2 args)
R.string.scheme_no_link = "No link available"
R.string.scheme_load_failed = "Failed to load schemes"
R.string.export_failed = "Export failed:"
R.string.data_error = "Data Error"
R.string.greeting_format = "Hello, %s!" (format string)
R.string.title_pm_kisan = "PM-KISAN Scheme"
R.string.title_soil_health = "Soil Health Card"
```

```
R.string.title_fasal_bima = "PM Fasal Bima Yojana"
```

APPENDIX: FIRESTORE SECURITY RULES SUMMARY

```
/farmers/{userId}      -> Owner-only CRUD. Create validates exact fields. Update cannot change o
/soilAnalysis/{docId} -> userId field must match auth UID for all operations.
/plantAnalysis/{docId}-> userId field must match auth UID.
/crops/{cropId}        -> Authenticated read only. No client writes.
/schemes/{schemeId}    -> Authenticated read only. No client writes.
/mandiPrices/{priceId}-> Authenticated read only. Updated by Cloud Functions.
/**                    -> Deny all (catch-all).
```

APPENDIX: ROOM DATABASE MIGRATION SEQUENCE

```
v1 -> v2: Remove FK constraints (early schema)
v2 -> v3: CREATE TABLE chat_messages
v3 -> v4: ALTER TABLE documents ADD COLUMN expiry_date INTEGER
v4 -> v5: ALTER TABLE soil_history ADD COLUMN user_id TEXT
          ALTER TABLE documents ADD COLUMN user_id TEXT
          ALTER TABLE chat_messages ADD COLUMN user_id TEXT
v5 -> v6: CREATE TABLE crop_schedules
v6 -> v7: CREATE TABLE plant_health
v7 -> v8: CREATE TABLE fields
          CREATE TABLE farm_tasks
          CREATE TABLE task_reminders
          CREATE TABLE task_logs
```

24. DEEP CODE — RecommendationActivity

Full Function Breakdown

onCreate() - Gets `analysis_id` from intent (if coming from HistoryActivity with specific record) or -1 (from ScanActivity -> uses latest) - Gets `DETECTED_SOIL_TYPE` string extra - Initializes TextToSpeech engine - Calls `loadFarmerProfile()` and `loadAnalysisData()` in parallel

loadFarmerProfile() - Fetches Firebase Auth email (shown directly — no Firestore needed) - Queries Firestore `farmers/{uid}` for `firstName` and `phone` - Populates `tvName`, `tvPhone`, `tvEmail` header fields

loadAnalysisData() - Runs on `dbExecutor` (`ExecutorService`) - If `analysisId != -1`: `soilDao().getAnalysisB` — specific record from History - Else: `soilDao().getLatestReportForUser(uid)` — most recent scan - Security check: `uid.equals(analysis.userId)` — prevents accessing another user's record even locally - Posts to main thread -> `populateForm(analysis)`

populateForm(SoilAnalysis analysis) - Parses `analysis.soilReportJson` as `JSONObject` - Key extraction: `json.optInt("N")`, `json.optInt("P")`, `json.optInt("K")`, `json.optDouble("pH", 6.5)` - Calls `setRow()` for each nutrient with threshold range

setRow(Textview vVal, TextView vRat, int val, int low, int high, String unit) Nutrient status thresholds used in the UI table:

Nitrogen: low=280, high=560 kg/ha
Phosphorus: low=10, high=25 kg/ha
Potassium: low=108, high=280 kg/ha

(Note: these UI display thresholds differ from the task suggestion thresholds — see `TaskSuggestionEngine`)

Color logic: `val < low OR val > high -> RED`; within range -> GREEN (`#2E7D32`)

calculateSoilHealthScore(double ph) — actual code:

```
float deviation = (float) Math.abs(ph - 7.0);  
int healthScore = Math.max(0, 100 - (int)(deviation * 15));
```

Score interpretation: - `>= 80`: Excellent (green `#2E7D32`) - `60-79`: Good (DKGRAY) - `40-59`: Needs Attention (orange `#FF9800`) - `< 40`: Poor (RED)

fetchProfessionalAdvice(int n, int p, int k, double ph, ...) - First checks `NetworkUtils.isNetworkAvailable()` — if offline, calls `buildFallbackAdvice()` immediately - Builds `AiModels.SoilAdvisoryRequest` with all fields including `languageCode` from `AppPreferences` - Calls `predictionRepository.fetchSoilAdvisory(request, callback)` - On success: checks `data.confidence < 45` -> appends `uncertaintyMessage` to advice - If `contextSummary` present in response: replaces the default context text - Calls `parseMarkdown()` to render basic markdown as HTML spans - Shows download button after advisory loaded - On success: triggers `TaskSuggestionEngine.suggestFromSoilAdvisory()` on `dbExecutor`

resolveBackendError(ApiEnvelope) Maps Firebase Functions error codes to user-facing messages:

```
UNAUTHENTICATED / PERMISSION_DENIED -> R.string.rec_error_backend_auth  
NOT_FOUND -> R.string.rec_error_backend_not_deployed  
UNAVAILABLE / DEADLINE_EXCEEDED -> R.string.rec_error_backend_unavailable  
RESOURCE_EXHAUSTED -> R.string.rec_error_backend_rate_limit  
default -> R.string.rec_error_connection
```

If `traceId` present in envelope: appends it to message (for debugging production failures)

buildFallbackAdvice(int n, int p, int k, double ph) Rule-based offline advice using exact same thresholds as `setRow()`: - `N < 280`: “Apply split urea doses with irrigation” - `N > 560`: “Reduce additional urea” - `P < 10`: “Add SSP/DAP near root zone” - `P > 25`: “Reduce phosphate application” - `K < 108`: “Apply MOP in split doses” - `K > 280`: “Hold potash, re-test later” - `pH < 6.0`: “Add lime” - `pH > 7.5`: “Add gypsum, improve organic matter”

parseMarkdown(String markdown) Custom lightweight markdown renderer (no library dependency):

```
html = markdown.replaceAll("\\*\\*(.*?)\\*\\*", "<b>$1</b>"); // bold  
html = html.replaceAll("####?\\s*(.*)", "<br><b>$1</b><br>"); // headings  
html = html.replaceAll("(?m)^-\\s+(.*)$", "&#8226; $1<br>"); // bullets  
html = html.replace("\\n", "<br>");  
return Html.fromHtml(html, Html.FROM_HTML_MODE_COMPACT);
```

savePdf() Uses Android's PdfDocument API (no third-party library): - A4 size: 595 × 842 points - Single page: PageInfo.Builder(595, 842, 1).create() - Calls drawAgriPass(canvas) for content - Saves to MediaStore.Files (Downloads folder) via ContentValues + getContentResolver().insert() - OutputStream in try-with-resources - Stores saved URI in savedPdfUri field for sharing

drawAgriPass(Canvas canvas) Hand-drawn PDF layout using Android Canvas API: 1. White background fill 2. Header: "MITTI MITRA - SOIL HEALTH CARD" centered at y=50 3. Non-affiliation note at y=70 4. Brand-green border rectangle (20, 20, 575, 822) 5. "FARMER DETAILS" section: name, location, date at y=120+ 6. Horizontal divider line (LTGRAY) 7. "SOIL ANALYSIS RESULTS" table with dark gray header row 8. Four data rows via drawRow(): N, P, K, pH with status colors 9. "SCIENTIFIC ADVISORY" section using StaticLayout for text wrapping (515px wide) 10. QR placeholder rectangle (gray box, "Scan to Verify" text)

drawRow(Canvas, param, val, status, statusColor, x, y) Draws one table row: parameter name at x+10, value at x+200, status at x+350 with color.

shareAsImage() Alternative sharing path — renders the same drawAgriPass() to a 595×842 Bitmap instead of PDF: - Creates ARGB_8888 Bitmap - Draws via Canvas - JPEG 95% quality saved to getCacheDir()/exports/soil_card.jpg - FileProvider URI -> ACTION_SEND with image/jpeg MIME

25. DEEP CODE — BackendFunctionsClient

Architecture

BackendFunctionsClient is the single point of entry for all Firebase Cloud Function calls. - Region: asia-south1 (Mumbai) — low latency for Indian users - Uses FirebaseFunctions.getInstance(REGION) — region-aware instance - Custom Gson with StringOrListTypeAdapter for flexible JSON parsing

Functions Exposed

```
getSoilAdvisory(SoilAdvisoryRequest, callback)
getPlantDiagnosis(PlantDiagnosisRequest, callback)
getCropSchedule(CropScheduleRequest, callback)
checkDuplicateAccount(DuplicateAccountRequest, callback)    // account merging
linkAccountIdentity(LinkIdentityRequest, callback)           // account merging
getFarmerChatResponse(ChatRequest, callback)
```

callEndpoint() — The Generic Dispatcher

```
private <T> void callEndpoint(String endpoint, Object payload, Type envelopeType, BackendCallback<T> callback) {
    Object callablePayload = toCallablePayload(payload); // POJO -> Map for Firebase
    functions.getHttpsCallable(endpoint).call(callablePayload)
        .addOnSuccessListener(result -> {
            ApiEnvelope<T> envelope = parseEnvelope(result, envelopeType);
            if (envelope.isSuccess()) callback.onSuccess(envelope);
            else callback.onFailure(envelope, null);
        })
        .addOnFailureListener(error -> {
```



```

        ApiEnvelope<T> envelope = mapThrowableToEnvelope(error);
        callback.onFailure(envelope, error);
    });
}

```

toCallablePayload()

Converts Java POJO to Firebase-compatible format:

```

JsonElement root = gson.toJsonTree(payload);    // POJO -> JsonElement
return gson.fromJson(root, Object.class);       // JsonElement -> Map/List/primitive

```

Firebase Functions `.call()` accepts Maps, Lists, and primitives — not arbitrary POJOs. Fallback: `Collections.emptyMap()` on any conversion error.

parseEnvelope()

Handles two response formats from backend: 1. **Standard:** {status, code, message, data, traceId} — new backend format 2. **Compat:** Raw data (no envelope wrapper) — legacy/third-party functions For compat: wraps in synthetic {status:"success", code:"OK", data: rawResponse} envelope.

mapThrowableToEnvelope()

Converts Firebase exception to `ApiEnvelope.error(code, message, traceId)`: - `FirebaseFunctionsException` uses `ex.getCode().name()` — e.g., "NOT_FOUND", "UNAVAILABLE" - Other: "UNKNOWN_ERROR"

StringOrListTypeAdapter

Custom Gson TypeAdapter for `List<String>` fields. AI backend sometimes returns "recommendation" (string) instead of ["recommendation"] (array). The adapter reads either format:

```

if (peek == BEGIN_ARRAY) -> parse as normal list
if (peek == STRING)      -> wrap single string in List
else                     -> skipValue(), return empty list

```

ApiEnvelope.java

Generic response wrapper:

```

public class ApiEnvelope<T> {
    public String status;    // "success" or "error"
    public String code;      // "OK", "NOT_FOUND", etc.
    public String message;   // Human-readable message
    public String traceId;   // Unique request ID for debugging
    public T data;           // Typed response data

    public boolean isSuccess() { return "success".equals(status); }
    public static <T> ApiEnvelope<T> success(T data) { ... }
    public static <T> ApiEnvelope<T> error(String code, String msg, String traceId) { ... }
}

```

26. DEEP CODE — FirebasePredictionRepository

Fallback Decision Logic

```
shouldUseLocalFallback(ApiEnvelope envelope):  
    -> ENABLE_LOCAL_AI_FALLBACK must be true (debug only)  
    -> GROQ_API_KEY must be non-empty  
    -> envelope.code must be: NOT_FOUND | UNAVAILABLE | DEADLINE_EXCEEDED | INTERNAL | UNKNOWN
```

This means: in release builds, there is NO local fallback. All AI goes through Cloud Functions. If Functions are down in release, user gets an error message.

Soil Advisory Path

```
fetchSoilAdvisory():  
    shouldPreferLocalFallbackForSoilAdvisory() -> always false (backend is primary)  
    -> functionsClient.getSoilAdvisory(request, callback)  
    -> No fallback for soil advisory (uses buildFallbackAdvice() in the Activity itself)
```

Plant Diagnosis Path (More Complex)

```
fetchPlantDiagnosis():  
    shouldPreferLocalFallbackForPlantDiagnosis() -> always false  
    -> functionsClient.getPlantDiagnosis(request, {  
        onSuccess -> callback.onSuccess  
        onFailure -> if shouldUseLocalFallback(envelope):  
            fetchPlantDiagnosisWithLocalGroq()  
        else:  
            callback.onFailure  
    })
```

fetchPlantDiagnosisWithLocalGroq()

Direct Groq API call with vision model: - Model: `meta-llama/llama-4-scout-17b-16e-instruct` (Llama 4 Scout — multimodal) - System prompt: “You are a plant pathologist for Indian agriculture. Return ONLY valid JSON with keys: `cropIdentified`, `healthStatus`, `confidence`, `issuesDetected`, `recommendations`, `uncertaintyMessage`, `rawJson`.” - User message: text part (location, language) + `image_url` part (base64 data URL) - Temperature: 0.1 (low — deterministic diagnosis) - Max tokens: 700

parseGroqDiagnosisResponse()

Handles LLM response quirks: 1. **Code fence stripping:** LLMs often wrap JSON in `json ...` — stripped before parsing 2. **JSON extraction:** finds first `{` to last `}` in response 3. **Dual key support:** `cropIdentified` or `crop_identified`, `healthStatus` or `health_status` 4. **Confidence normalization:** - If confidence is 0-1 range (e.g., 0.87): multiply by 100 - If 0: default to 50 - If < 0 or > 100: clamp

extractFirstJsonObject(String rawText)

```
if rawText starts with "```":
    strip from first newline to last "```"
find first '{' and last '}'
return substring between them
```

Handles LLM tendency to wrap JSON in markdown code fences.

27. DEEP CODE — TaskSuggestionEngine

Exact Thresholds for Task Creation

suggestFromSoilAdvisory() triggers task creation if ANY condition true:

```
nitrogen < 200 || phosphorus < 12 || potassium < 120 || ph < 6.0 || ph > 7.8
```

Creates ONE “Soil Correction Task”: - Due: now + 48 hours - Priority: 1 (HIGH) - Source: “soil_advisory” - Auto-reminder: 12 hours before due (36 hours from now)

suggestFromPlantDiagnosis() triggers if:

```
healthStatus != null && !"Healthy".equalsIgnoreCase(healthStatus)
```

Creates “Disease Follow-up Check” task: - Due: now + 24 hours - Priority: 1 (HIGH) - Reminder: 12 hours from now (midpoint)

suggestFromCropSchedule() behavior: - If AI returned schedule items: one task per `ScheduleItem` with `dueAt = now + (week * 7days)` - If schedule empty: uses `FarmTaskTemplates.forCrop(userId, cropName, now)` — static hardcoded templates - Reminder: 6 hours before each task due date

createReminder() safety floor:

```
reminder.remindAt = Math.max(remindAt, System.currentTimeMillis() + 60_000L);
```

Reminder is always at least 1 minute in the future — prevents past-due reminders.

28. DEEP CODE — AiModels DTOs (Full)

SoilAdvisoryRequest (sent to Cloud Function)

```
int nitrogen;          // kg/ha
int phosphorus;        // kg/ha
int potassium;          // kg/ha
double ph;              // 0-14
String userNotes;       // Free text from scan screen
String weather;         // "[cloud] Overcast 28°C | 75%"
String moisture;        // "0.23 m³/m³"
String detectedSoil;    // "Black", "Red", etc.
String languageCode;    // "hi", "ta", "en", etc.
String location;        // "Pune, Maharashtra"
```

SoilAdvisoryData (received from Cloud Function)

```
String advisoryMarkdown;    // Markdown text with fertilizer advice
String contextSummary;      // Brief context line shown above advisory
Integer confidence;         // 0-100, if < 45 shows uncertaintyMessage
String uncertaintyMessage;  // Warning text for low confidence
Map<String, Object> metadata; // Extra data (reserved)
```

PlantDiagnosisRequest

```
String imageBase64DataUrl; // "..."
String languageCode;
String location;
```

PlantDiagnosisData

```
String cropIdentified;      // "Rice", "Wheat", etc.
String healthStatus;        // "Healthy", "Diseased", "Stressed"
Integer confidence;         // 0-100
List<String> issuesDetected; // ["Late Blight", "Nutrient Deficiency"]
List<String> recommendations; // ["Apply Mancozeb", "Add NPK"]
String uncertaintyMessage;
String rawJson;             // Full model response stored for debugging
```

CropScheduleRequest

```
String cropName;           // "Paddy", "Wheat", etc.
String location;           // For regional calendar advice
String currentMonth;       // "May" - planning starting point
String languageCode;
```

CropScheduleData

```
String crop;
String bestPlantingMonth;
String bestHarvestMonth;
Integer durationDays;
String fertilizerSchedule; // Text summary
String irrigationTips;
String pestWatch;
Integer confidence;
String uncertaintyMessage;
List<ScheduleItem> schedule; // Ordered list of activities
```

ScheduleItem

```
Integer week;           // Week number from planting (1, 2, 4, 8, ...)
String activity;        // "Apply basal dose fertilizer"
String tips;            // "Use 50kg urea per acre"
String dueDate;         // Optional absolute date string
```

ChatRequest

```
String prompt;           // System context
String userQuery;        // What farmer asked
String languageCode;
String location;
String soilContext;      // JSON string of last soil scan (for context)
```

ChatResponseData

```
String replyMarkdown;
Integer confidence;
String uncertaintyMessage;
```

29. DEEP CODE — NotificationWorker Exact Values

Notification IDs

```
ID 1: Weather alert notification
ID 2: Mandi price notification
ID 100-2099: Document expiry -> (int)(doc.documentId % (MAX_INT - 100)) + 100
ID 2000-2100+: Task reminders -> (int)((reminder.id % 100000) + 2000)
```

Weather Thresholds (actual code)

```
if (humidity > 80) msg += getString(R.string.weather_high_humidity); // fungal risk
else if (temp > 35) msg += getString(R.string.weather_high_heat);    // irrigation needed
```

Note: No AND — only first matching condition fires.

Location Fallback

```
double lat = Double.parseDouble(locPrefs.getString("last_lat", "20.5937"));
double lon = Double.parseDouble(locPrefs.getString("last_lon", "78.9629"));
```

Default: 20.5937°N, 78.9629°E — geographic center of India (Nagpur area). Used when user has never opened ScanActivity.

HTTP Call in Worker

NotificationWorker uses **synchronous** Retrofit call:

```
.getAgroWeather(lat, lon).execute().body()
```

This is correct — `Worker.doWork()` runs on a background thread managed by `WorkManager`. Synchronous HTTP is fine here (unlike Activities where you'd need `.enqueue()`).

Sunrise/Sunset Parsing

```
SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm", Locale.getDefault());
Date sunrise = sdf.parse(sunriseStr); // e.g., "2026-05-06T06:12"
```

```
appPrefs.setLastSunrise(sunrise.getTime()); // saves Unix millis
```

BaseActivity reads these to auto-switch dark mode at sunset.

Notification Channel

```
NotificationChannel channel = new NotificationChannel(
    "mitti_mitra_alerts",
    "Farming Alerts",
    NotificationManager.IMPORTANCE_DEFAULT
);
channel.setDescription("Alerts about weather, market prices, documents, and tasks");
```

Channel created on every `triggerNotification()` call — `createNotificationChannel()` is idempotent (safe to call multiple times).

PendingIntent Flags

```
PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
```

`FLAG_IMMUTABLE` required on Android 12+. `FLAG_UPDATE_CURRENT` updates existing pending intent with new extras if same request code used.

30. COMPLETE FILE COUNT AND PROJECT STATISTICS

Category	Count
Java source files	97
Layout XML files	50+
Activities	26
Room entities (DB tables)	9
Room DAOs	9
DB migrations	6 (v2->v3 through v7->v8)
Firestore collections	6
Retrofit service interfaces	6
Domain repository interfaces	3
Repository implementations	3
Utility classes	12+
Supported languages	14
String resources (approx)	300+
Firebase Cloud Functions	6 endpoints
WorkManager workers	1 (NotificationWorker)
TFLite models	1 (soil_classifier.tflite)
ML classes (TFLite output)	10 soil types
AI models used	2 (Llama 3.3 70B, Llama 4 Scout 17B)
Min Android SDK	24 (Android 7.0)
Target Android SDK	36 (Android 14)
App version	1.0.5 (versionCode 5)

Category	Count
Firebase project	mitti-mitra-4ee33

31. SOIL SCIENCE BEHIND THE APP

Why These Nutrients Matter

Nitrogen (N): Primary macronutrient. Essential for leaf/stem growth, chlorophyll production. Deficiency: yellowing leaves (chlorosis). Excess: lush growth, delayed fruiting, pest attraction.

Phosphorus (P): Root development, flowering, fruiting, energy transfer (ATP). Deficiency: purple/reddish leaves, poor root system. Often “fixed” in acidic soils (iron phosphate) — Red and Laterite soils.

Potassium (K): Disease resistance, water regulation, protein synthesis, fruit quality. Deficiency: leaf edge browning (scorch). Clay soils hold K well; Sandy soils lose it to leaching.

pH: Controls nutrient availability. At $\text{pH} < 6.0$, aluminium and manganese become toxic; P gets locked. At $\text{pH} > 7.5$, iron, zinc, manganese become unavailable. Sweet spot: 6.0–7.5 for most Indian crops.

ISRIC SoilGrids Data Quality

SoilGrids uses machine learning trained on 150,000+ soil profiles worldwide combined with remote sensing data. For India, additional data from ICAR (Indian Council of Agricultural Research) is incorporated. Resolution: $250\text{m} \times 250\text{m}$ grid. Uncertainty quantified as Q0.05 and Q0.95 (5th and 95th percentile predictions).

Why Corrections Are Needed

SoilGrids gives statistical means over 250m cells. A farmer’s field may differ because: - Soil type visual identification gives local evidence - Organic matter (brightness) varies within cells - Past fertilizer application history (user notes) - Microclimate effects not captured at 250m resolution

Our correction factors bridge this gap between satellite estimates and field reality.

32. INTERVIEW / VIVA ADDITIONAL QUESTIONS

Q: Explain the TFLite preprocessing in detail. A: The model expects float32 input of shape [1, 224, 224, 3]. We resize the bitmap to 224×224 , then extract pixel values using `getPixels()`. Each pixel is an ARGB int — we shift right by 16 bits to get R, by 8 bits for G, and mask with 0xFF for B. Each channel is divided by 255.0f to normalize to [0,1]. Values are packed into a direct ByteBuffer in native byte order (required by TFLite). Output is [1, 10] — we take argmax.

Q: What model architecture is `soil_classifier.tflite` likely using? A: Almost certainly a MobileNet or EfficientNet-Lite variant. These are compact CNNs optimized for mobile inference. Mo-

bileNetV2 at 224×224 input is the standard TFLite baseline for image classification tasks. The 10-class output matches our soil type count.

Q: How does Firebase Functions region affect performance? A: Cloud Functions deployed to `asia-south1` (Mumbai) are physically closer to Indian users than the default `us-central1`. Network latency for a Mumbai user hitting Mumbai Functions vs US Functions: ~15ms vs ~200ms. For AI calls that take 2-5 seconds, this 185ms improvement matters less, but it adds up across all function invocations.

Q: What is the TypeToken pattern used in Gson? A: Java’s type erasure removes generic type information at runtime. `List<String>` becomes just `List` in bytecode. Gson needs the full type to deserialize correctly. `TypeToken.getParameterized(List.class, String.class).getType()` captures the generic type at compile time by creating an anonymous subclass, preserving the type parameter.

Q: Why use MappedByteBuffer for TFLite model loading? A: `MappedByteBuffer` memory-maps the file — the OS maps the file bytes directly into virtual memory without copying to heap. This is zero-copy: the model file on disk IS the buffer. Advantages: no heap allocation for the model file, OS handles paging in/out, faster than reading to a byte array. The model stays accessible as long as the file descriptor is open.

Q: Explain double-checked locking in MittiMitraDatabase. A: The outer `if (INSTANCE == null)` check avoids locking on every call (performance). If null, we enter `synchronized` to prevent two threads from both passing the first check. Inside sync, we check again (`if (INSTANCE == null)`) because another thread might have created the instance while we waited for the lock. `volatile` ensures the fully-constructed object is visible across CPU caches.

Q: What is the difference between minifyEnabled and shrinkResources? A: `minifyEnabled true` runs ProGuard/R8 which removes unused Java classes and methods, and obfuscates names. `shrinkResources true` removes unused Android resources (layouts, drawables, strings). Both require each other — `shrinkResources` only works when `minifyEnabled` is true (it needs the class analysis to know which resources are referenced).

Q: How does the Groq API handle Indian language responses? A: The `languageCode` field in `SoilAdvisoryRequest` is sent to the Cloud Function, which includes it in the Llama 3.3 70B system prompt: “Respond in [language_code] language.” Llama 3.3 70B supports Hindi, Tamil, Telugu, Kannada, Marathi, Bengali, Gujarati, and Punjabi natively. For northeast languages (Mizo, Bodo, Garo, Manipuri), the model may not respond accurately — those users likely get English responses.

Q: What happens if the TFLite model file is corrupted or missing? A: `loadModelFile()` throws `IOException` if the file cannot be opened. `runLocalInference()` catches this in the outer try-catch (`Exception e`) block and calls `generateSmartReport("Analysis Error", userNotes)`. The report is still generated and saved — with soil type “Analysis Error” — so the user can still get weather/GPS-based data even without ML classification.

Q: Why is the WorkManager minimum interval 15 minutes? A: Android’s Doze mode and battery optimization restrict background work. WorkManager respects this — the minimum `PeriodicWorkRequest` interval is 15 minutes (enforced by the framework, even if you set less). This is a platform limitation, not an app choice.

Q: What is ExistingPeriodicWorkPolicy.KEEP vs REPLACE? A: `KEEP` — if a work with the same unique name already exists and is not cancelled, do nothing. New request is ignored. `REPLACE` — cancel existing work and replace with new. We use `KEEP` to prevent losing the notification schedule

timing when the app is opened multiple times. REPLACE would reset the 15-minute interval each time the app opens, potentially delaying notifications.

Q: How does the app handle the case where Room DB version is ahead of the migrations?

A: If a device has DB version 9 (hypothetical future) and the code only has migrations up to v8, Room throws `IllegalStateException: A migration from 8 to 9 was required but not found`. The app crashes on launch. This is why we removed `fallbackToDestructiveMigration()` — it would silently wipe data instead of crashing, which is worse. The correct fix is always providing proper migrations.

APPENDIX: KEY SHARED PREFERENCES FILES

File Name	Used By	Contents
MittiMitraSession	SessionManager	userId, userName, isGuest
MittiMitraPrefs	AppPreferences	theme, language, font, GPS, crop, soil
MittiMitra_Notifications	NotificationWorker, NotificationSettingsActivity	notif_weather, notif_mandi
scan_cache	ScanActivity, NotificationWorker	booleans
mandi_price_cache	MandiActivity, NotificationWorker	weather string, soil_dyn, loc name
user_location_cache	ScanActivity, NotificationWorker	last_prices JSON, last_commodity
		last_lat, last_lon as Strings

APPENDIX: ASSETS DIRECTORY

File	Type	Purpose
soil_classifier.tflite	TFLite model	On-device soil type classification
offline_tips.json	JSON	Fallback agricultural tips when no network
schemes_data.json	JSON	Government scheme data (offline-first)
Soil_data.csv	CSV	Reference soil property data

APPENDIX: NOTIFICATION ID ALLOCATION TABLE

Range	Source	Logic
1	Weather alert	Fixed
2	Mandi prices	Fixed

Range	Source	Logic
100–2099	Document expiry	(doc.documentId % (MAX_INT-100)) + 100
2000–102000	Task reminders	(reminder.id % 100000) + 2000

MittiMitra Ultimate Guide — Team A-14, Amrita Vishwa Vidyapetham Generated from full codebase analysis — Complete Edition, 2026